Engineering Applications of Artificial Intelligence 164 (2026) 113288

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

OPHEP-Miner: One-phase high-efficiency pattern mining utilizing tree structures

Xuan Liu [a], Genlang Chen [a], Fangyu Wu [b,*], Wanli Zuo [c], Youxi Wu [d]

[a] School of Computer and Data Engineering, NingboTech University, China
[b] School of Advanced Technology, Xi'an Jiaotong-Liverpool University, China
[c] School of Mechanical Engineering and Mechanics, Ningbo University, China
[d] School of Artificial Intelligence, Hebei University of Technology, China

ARTICLE INFO

Keywords:
High-utility pattern mining
High-efficiency pattern
Tree structure
Pruning strategy

ABSTRACT

High-efficiency pattern mining aims to identify patterns generating substantial utility while minimizing costs. The existing mining methods mainly employ the pattern combination strategy or projection technology to generate high-order patterns, thereby resulting in a large number of candidate patterns. Although the generation of invalid patterns can be avoided by storing the transaction dataset in a tree structure for mining, this approach is considered time-consuming. To address this issue, we employed the prefix tree and utility vector structure to discover high-efficiency patterns within one phase. To further enhance mining efficiency, a bottom-up single-path pattern enumeration method was designed, which eliminates the need for complex tree structure constructions. Additionally, three pruning strategies were devised to limit the pattern search space, including a novel pruning technique specifically designed for single-path scenarios. The effectiveness of the proposed algorithm has been validated through comparative analyses conducted on eight diverse datasets using five algorithms. The results show that the proposed algorithm outperforms other approaches, particularly due to its effective pruning techniques, which enable it to run up to 64 times faster than the best competing algorithm in some cases. Furthermore, we used a movie dataset as a case study to illustrate the applicability of our proposed approach.

1. Introduction

Pattern mining is a data mining technique to discover significant and frequently occurring patterns or rules in large datasets. These patterns can be itemsets (Liu et al., 2023; Song et al., 2024), sequences (Geng et al., 2023; Li et al., 2025; Sun et al., 2024), subgraphs (Khalil et al., 2024) or other structured data formats (Ho et al., 2025; Jazayeri and Yang, 2024). Pattern mining has diverse applications in market analysis, recommendation systems, bioinformatics, and so on (Iqbal et al., 2022; Liu et al., 2022b; Zhang et al., 2022). By mining patterns, hidden associations, trends, anomalies, or other valuable insights can be extracted from the data, enabling tasks such as decision-making, predictive analysis, and business optimization (Dong et al., 2020). Frequent pattern mining specifically focuses on identifying patterns or itemsets that occur frequently in a dataset (Chee et al., 2019). There are numerous pattern types based on support, such as tri-patterns (Min et al., 2020), frequent episodes (Zhu et al., 2025) and pyramid scheme patterns (Wang et al., 2021). However, frequent itemset mining techniques have limitations that make them unsuitable for certain real-world datasets and applications. They assume the maximum occurrence of each item in a transaction is one and consider all items equally important. In reality, transactions may have multiple occurrences of items, and different items may have varying profitability or importance. To address this issue, High-Utility Pattern Mining (HUPM) has been proposed.

HUPM aims to discover itemsets with high utility, considering multiple occurrences of items and their varying importance. This approach provides a more flexible and realistic analysis of transactional datasets. HUPM has garnered significant scholarly interest, leading to extensive research in various areas such as high utility co-location pattern mining (Wang et al., 2023; Yang et al., 2024), high utility sequential pattern mining (Sun et al., 2024; Zhang et al., 2023a), skyline frequent-utility pattern mining (He et al., 2024; Liu et al., 2022a) and low utility pattern mining (Zhang et al., 2025). In the mentioned research, users can assign appropriate utility values to items based on specific application scenarios, such as sales profit, customer satisfaction, or page viewing time. Patterns with higher utility values are generally more

* Corresponding author. No. 111, Ren'ai Road, Dushu Lake Science and Education Innovation District, Suzhou Industrial Park, Suzhou, Jiangsu Province, China.
E-mail address: fangyu.wu02@xjtlu.edu.cn (F. Wu).

https://doi.org/10.1016/j.engappai.2025.113288
Received 5 May 2025; Received in revised form 30 September 2025; Accepted 19 November 2025
Available online 24 November 2025
0952-1976/© 2025 Published by Elsevier Ltd.

valuable to users (Gan et al., 2021). However, practical situations also incorporate metrics related to costs. For instance, in smart tourism recommendations, users aspire to solutions that not only offer cost-effectiveness but also ensure high popularity. Therefore, High-Efficiency Pattern Mining (HEPM) was proposed to simultaneously consider both cost and utility factors, aiming to discover patterns that are both cost-effective and high in utility (Zhang et al., 2023b). Although there have been research results on low-cost pattern mining, such as correlated and low-average-cost pattern mining (Liu et al., 2024) and low-average-cost and frequent pattern mining (Fournier-Viger et al., 2020a), these mining algorithms consider two metrics simultaneously. For example, correlated and low-average-cost pattern mining requires the mined patterns to have an average cost lower than a given cost threshold and a correlation higher than a given correlation threshold. Because these algorithms consider two interest metrics separately, it is not possible to rank the mined patterns according to user interest. However, HEPM algorithms integrate cost and utility into a single interest factor, enabling the sorting of mined patterns based on their level of interest. This approach helps users identify the most valuable patterns, thereby better meeting user requirements. The state-of-the-art HEPM algorithms are HEPMiner, MHEP and EHEPM. HEPMiner and EHEPM are derived from the utility list (Liu and Qu) approach, while MHEP is based on the EFIM (Zida et al., 2017) algorithm. Although these algorithms are classic, they still produce a considerable number of unpromising patterns during candidate pattern generation. While utilizing a tree structure to store a transactional dataset can partially alleviate this issue, the recursive mining of pattern trees and the creation of local transaction trees consume a substantial amount of time. To tackle this challenge, we have introduced a one-phase mining algorithm based on the TV (prefix Tree and utility Vector) structure that incorporates an optimized enumeration strategy and effective pruning strategies. The main contributions of this paper include the following aspects.

- A bottom-up single-path pattern enumeration method was proposed to avoid the generation of massive TV structures.
- Three pruning strategies were devised to restrict the exploration space. Specifically, a novel pruning approach was devised to optimize mining effectiveness for trees that follow a single path.
- Various datasets were used in experiments to verify the effectiveness of the algorithm proposed in this paper in terms of runtime, space utilization, the number of candidate patterns, and scalability.
- A case study illustrates the application of our proposed OPHEP-Miner algorithm for generating recommendations based on the movie dataset.

The remainder of this paper is organized as follows. Section 2 provides an overview of the current research. Section 3 introduces the fundamentals of HEPM and presents the problem model. In Section 4, the key techniques of the OPHEP-Miner algorithm are introduced. The experimental process is detailed in Section 5, along with an analysis of the experimental results. Finally, Section 6 concludes the paper and outlines future work.

2. Related work

2.1. High-utility pattern mining

HUPM techniques are mainly categorized into four types: Apriori-based algorithms, tree-based methods, utility-list methods and projection-based algorithms. Apriori-based algorithms employ a breadth-first search strategy to progressively create candidate itemsets. Nonetheless, the drawback of these methods lies in their potential inefficiency due to the necessity of assessing numerous itemset combinations that are absent in the database (Lan et al., 2014). To address these issues, tree-based methods have been proposed. These methods store transaction databases in tree structures and mine patterns by recursively constructing subtrees. IHUP (Ahmed et al., 2009), UP-growth (Tseng et al. 2010), and UP-growth+ (Tseng et al., 2012) are two-phase algorithms, requiring an additional pass through the dataset to calculate the actual utility values of patterns. The Hamm algorithm introduces a utility pool based on tree structures to achieve one-phase pattern mining (Qu et al., 2023). In addition to the above two types of methods, utility list-based algorithms are another common approach for mining high-utility patterns. HUP-Miner (Krishnamoorthy, 2015), mHUIMiner (Peng et al.,), ULB-Miner (Duong et al., 2018),UBP-Miner (Wu et al., 2022) and FOTH (Yan et al., 2024) are all based on the utility-list structure. These methods generate larger patterns by connecting patterns through utility lists, avoiding the need for repetitive dataset traversals. The utility list structure is widely used and has been extended to mine other types of patterns (Gan et al., 2019; Han et al., 2024; Kim et al., 2021). Projection-based algorithms improve efficiency by recursively mining patterns through the construction of projected databases that support patterns. EFIM (Zida et al., 2017) and HUI-PR (Wu et al., 2019) algorithms are projection-based HUPM mining algorithms. This technique can also be used to solve other utility-oriented pattern mining problems (Nguyen et al., 2021; Singh and Biswas, 2024). In addition, some other algorithms have been proposed. d2HUP (Liu et al., 2012), IMHUP (Ryang and Yun, 2017), and REX (Qu et al., 2020) are hyperlink-based algorithms that employ pointers for traversing transactions. R-Miner is based on the concept of residual utility and designs two structures for HUPM (Sra and Chand, 2024b). Additionally, they also utilized this technology to achieve incremental data mining (Sra and Chand, 2024a). In summary, HUPM algorithms based on various technologies have been proposed and have been extensively explored.

2.2. High-efficiency pattern mining

For utility-oriented pattern mining, the traditional definition of utility primarily refers to profit, which contains a positive implication where patterns with higher utility values are considered more favourable. This perspective tends to ignore critical factors like costs and investments. To address this limitation, the concept of low-cost pattern mining has emerged (Wu and Huang, 2011). Subsequently, many improved low-cost pattern mining techniques were proposed (Fournier-Viger et al., 2020a, 2020b; Liu et al., 2024; Truong et al., 2022). HEPM aims to extract patterns that generate high utility at low investment by incorporating investment and utility metrics. HEPMiner is the first mining approach for HEPM, which is based on the well-known utility list framework (Zhang et al., 2023b). EHEPM further enhances performance by refining pruning strategies (Yildirim, 2025). MHEI is an extension of the EFIM algorithm for discovering high-efficiency patterns (Huynh et al., 2024). In addition, MHEI utilizes two tighter efficiency upper bounds, namely, *ssef* and *slef*, to design pruning strategies. Both the utility list framework and the EFIM algorithm have been extensively expanded for solving utility-driven pattern mining problems. Although these algorithms achieve satisfactory performance, we observe that an excessive number of candidate patterns are generated when processing dense databases. To address this issue, we propose employing the TV-structure for mining, a framework originally introduced for mining high-utility patterns (Qu et al., 2023). Accordingly, this paper modifies the algorithm to enable HEPM and introduces three novel pruning strategies.

3. Preliminaries

In this section, we provide a formal definition of the HEPM problem (Yang et al., 2024; Zhang et al., 2023b).

A transaction dataset D comprises multiple transactions. Each transaction $\mathbf{t}_f$ has a unique identification number f, known as a transaction identifier (TID) and consists of a set of items. Each item i_j in $\mathbf{t}_f$ has an internal utility denoted as iu (i_j, $\mathbf{t}_f$), where i_j belongs to the set of all items in D, $I = \{i_1, i_2, \ldots, i_n\}$. Furthermore, each item i_j is associated with

a positive value eu (i_j), indicating its unit profit or external utility. Moreover, i_j is assigned a unit investment represented by $ui(i_j)$, which can take various forms such as time, money, distance, or other factors. A pattern or itemset refers to any subset of set I. Specifically, an itemset containing k items is known as a k-itemset. Table 1 illustrates a transaction dataset D, and Table 2 shows the external utility and investment of each item in D.

Definition 1. (Utility of a pattern) Suppose we have a pattern $\mathbf{p}$. The utility of $\mathbf{p}$ in transaction $\mathbf{t}_f$ and dataset D are defined as $u_t(\mathbf{p},\mathbf{t}_f) = \sum_{\forall j\in\mathbf{p}} eu(j)\times iu(j,\mathbf{t}_f)$ and $u_d(\mathbf{p}) = \sum_{\mathbf{t}_f\in D\wedge\mathbf{p}\subseteq\mathbf{t}_f} u_t(\mathbf{p},\mathbf{t}_f)$, respectively.

Example 1. According to Tables 1 and iu(b,$\mathbf{t}_1$) = 2 and iu(c,$\mathbf{t}_1$) = 3. According to Table 2, eu(b) = 3 and eu(c) = 2. Thus, $u_t(\{b,c\},\mathbf{t}_1) = 2\times3 + 3\times2 = 6+6 = 12$. Moreover, $u_d(\{b,c\}) = u_t(\{b,c\},\mathbf{t}_1) + u_t(\{b,c\},\mathbf{t}_3) + u_t(\{b,c\},\mathbf{t}_5) = 12+17+16 = 45$.

Definition 2. (Investment) The investment of item j in transaction $\mathbf{t}_f$ and dataset D are defined as $inv_t(j,\mathbf{t}_f) = ui(j)\times iu(j,\mathbf{t}_f)$ and $inv_d(j) = \sum_{j\in\mathbf{t}_f} inv_t(j,\mathbf{t}_f)$, respectively. The investment of pattern $\mathbf{p}$ in dataset D is defined as $inv_d(\mathbf{p}) = \sum_{j\in\mathbf{p}} inv_d(j)$.

Example 2. According to Tables 1 and 2, ui(c) = 5 and iu(c,$\mathbf{t}_1$) = 3. Hence, $inv_t(c,\mathbf{t}_1) = 5\times 3 = 15$. Similarly, $inv_d(c) = inv_t(c,\mathbf{t}_1) + inv_t(c,\mathbf{t}_2) + inv_t(c,\mathbf{t}_3) + inv_t(c,\mathbf{t}_5) = 15+15+5+10 = 45$. Moreover, $inv_d(a) = 8$, $inv_d(b) = 33$, $inv_d(d) = 20$, and $inv_d(e) = 72$. Hence, $inv_d(\{a,c\}) = inv_d(a) + inv_d(c) = 8+45 = 53$.

Definition 3. (Efficiency) The efficiency of item j and pattern $\mathbf{p}$ in transaction $\mathbf{t}_f$ are defined as $e_t(j,\mathbf{t}_f) = \frac{u_t(j,\mathbf{t}_f)}{inv_d(j)}$ and $e_t(\mathbf{p},\mathbf{t}_f) = \frac{u_t(\mathbf{p},\mathbf{t}_f)}{inv_d(\mathbf{p})}$, respectively. The efficiency of pattern $\mathbf{p}$ in dataset D is defined as $e_d(\mathbf{p}) = \sum_{\mathbf{t}_f\in D\wedge\mathbf{p}\subseteq\mathbf{t}_f} e_t(\mathbf{p},\mathbf{t}_f) = \frac{\sum_{\mathbf{t}_f\in D\wedge\mathbf{p}\subseteq\mathbf{t}_f} u_t(\mathbf{p},\mathbf{t}_f)}{inv_d(\mathbf{p})} = \frac{u_d(\mathbf{p})}{inv_d(\mathbf{p})}$.

Example 3. In Example 2, $e_t(a,\mathbf{t}_1) = \frac{u_t(a,\mathbf{t}_1)}{inv_d(a)} = \frac{1}{8} = 0.125$. Moreover, $e_t(\{a,e\},\mathbf{t}_1) = \frac{u_t(\{a,e\},\mathbf{t}_1)}{inv_d(\{a,e\})} = \frac{9}{80} = 0.1125$, and $e_d(\{a,e\}) = e_t(\{a,e\},\mathbf{t}_1) + e_t(\{a,e\},\mathbf{t}_4) + e_t(\{a,e\},\mathbf{t}_5) = \frac{9}{80}+\frac{14}{80}+\frac{17}{80} = \frac{40}{80} = 0.5$.

Definition 4. (High-efficiency pattern) If the efficiency $e_d(\mathbf{p})$ of pattern $\mathbf{p}$ in dataset D is greater than or equal to the minimum efficiency threshold value ε, then $\mathbf{p}$ is a High-Efficiency Pattern (HEP).

Problem statement: The problem of HEPM involves finding all HEPs.

Example 4. If $\varepsilon = 0.5$, then pattern $\mathbf{p} = \{a, e\}$ is an HEP according to Example 3. All HEPs in Table 1 are $\{\{a\},\{a,b\},\{a,e\},\{b\},\{b,c\},\{e\}\}$.

4. Proposed method

In this paper, we propose a one-phase algorithm named OPHEP-Miner for the fast discovery of HEPs based on TV structures. The framework of OPHEP-Miner is shown in Fig. 1. In the following, we will provide a description of our proposed algorithm and the techniques employed.

4.1. Data structure and pruning strategies

Our proposed OPHEP-Miner algorithm utilizes the TV structure for storing datasets. This structure was initially presented in the Hamm algorithm (Qu et al., 2023). Below are the fundamental definitions necessary for constructing a TV structure.

Table 1
Transaction dataset D.

TID	Items	Internal Utility
$\mathbf{t}_1$	*a b c d e*	1 2 3 2 2
$\mathbf{t}_2$	*c d e*	3 2 1
$\mathbf{t}_3$	*b c*	5 1
$\mathbf{t}_4$	*a e*	2 3
$\mathbf{t}_5$	*a b c d e*	5 4 2 1 3

Table 2
External utility and investment of items.

Item	*a*	*b*	*c*	*d*	*e*
External Utility (eu (i_j))	1	3	2	1	4
Unit Investment (ui (i_j))	1	3	5	4	8

Definition 5. (Transaction utility) The transaction utility of transaction $\mathbf{t}_f$ is defined as $tu(\mathbf{t}_f) = \sum_{j\in\mathbf{t}_f} u_t(j,\mathbf{t}_f)$.

Example 5. According to Tables 1 and 2, tu($\mathbf{t}_3$) = 5×3 + 1×2 = 17. Similarly, tu($\mathbf{t}_1$) = 23, tu($\mathbf{t}_2$) = 12, tu($\mathbf{t}_4$) = 14, and tu($\mathbf{t}_5$) = 34.

Definition 6. (Transaction weighted utility) The transaction weighted utility of pattern $\mathbf{p}$ is defined as $twu(\mathbf{p}) = \sum_{\mathbf{p}\subseteq\mathbf{t}_f} tu(\mathbf{t}_f)$.

Example 6. $twu(\{a,e\}) = tu(\mathbf{t}_1) + tu(\mathbf{t}_4) + tu(\mathbf{t}_5) = 23+14+34 = 71$, since pattern $\{a, e\}$ occurs in $\mathbf{t}_1$, $\mathbf{t}_4$, and $\mathbf{t}_5$. Similarly, we know the twu values of all items in D shown in Table 3.

Definition 7. (Remaining utility) The remaining utility of pattern $\mathbf{p}$ in transaction $\mathbf{t}_f$ and dataset D are defined as $ru_t(\mathbf{p},\mathbf{t}_f) = \sum_{j\in\mathbf{t}_f\wedge\mathbf{p}\prec j} u_t(j,\mathbf{t}_f)$ and $ru_d(\mathbf{p}) = \sum_{\mathbf{p}\subseteq\mathbf{t}_f\wedge\mathbf{t}_f\in D} ru_t(\mathbf{p},\mathbf{t}_f)$, respectively. For two patterns, $\mathbf{p}$ and $\mathbf{q}$, if $\mathbf{p}\prec\mathbf{q}$, then $\mathbf{p}$ precedes $\mathbf{q}$ based on the total order of items utilized by the mining process. The total order can be alphabetical or twu ascending.

Example 7. According to Tables 1 and 2, $ru_t(\{b,c\},\mathbf{t}_1) = u_t(d,\mathbf{t}_1) + u_t(e,\mathbf{t}_1) = 2+8 = 10$. Moreover, $ru_d(\{b,c\}) = ru_t(\{b,c\},\mathbf{t}_1) + ru_t(\{b,c\},\mathbf{t}_3) + ru_t(\{b,c\},\mathbf{t}_5) = 10+0+13 = 23$. If the total order of items is alphabetical, we have $a\prec b$, it means that a precedes b.

Definition 8. (Transaction-weighted efficiency) The transaction-weighted efficiency of pattern $\mathbf{p}$ in dataset D is defined as $twe(\mathbf{p}) = \frac{twu(\mathbf{p})}{inv_d(\mathbf{p})}$.

Example 8. In Example 6, the twu value of {a,e} is 71. According to Definition 8, $twe(\{a,e\}) = \frac{twu(\{a,e\})}{inv_d(\{a,e\})} = \frac{71}{80} = 0.8875$.

The transaction-weighted efficiency is an upper bound on the efficiency of a pattern. It was originally introduced in the HEPMiner algorithm (Zhang et al., 2023b) and is further utilized in the MHEI algorithm (Huynh et al., 2024) for mining HEPs. This upper bound design draws inspiration from the well-known transaction-weighted utility introduced in the Two-Phase algorithm (Liu et al., 2005). The creation of this upper bound is motivated by the need to satisfy the Apriori property (1994Agrawal and Srikant,).

Theorem 1. *For a pattern $\mathbf{p}$, if $twe(\mathbf{p}) < \varepsilon$, then for any pattern $\mathbf{q}\supseteq\mathbf{p}$, it holds that $\mathbf{e}_d(\mathbf{p}) < \varepsilon$.*

Theorem 1 has been proved in the HEPMiner algorithm (Zhang et al., 2023b). We omit its repetition herein.

Pruning Strategy 1. For a pattern $\mathbf{p}$, if $twe(\mathbf{p}) < \varepsilon$, then $\mathbf{p}$ and all its supersets are not HEPs.

In our OPHEP-Miner algorithm, Pruning Strategy 1 is utilized to filter out unpromising items. Subsequently, the retained items are employed to build the prefix tree structure for storing the transaction dataset.

4.1.1. Components of the TV structure

Based on the aforementioned concepts, we introduce the TV structure. Each TV structure is associated with a prefix pattern $\mathbf{p}$. Initially, $\mathbf{p}$ is the empty set. Subsequently, for each item i in a TV structure with its prefix itemset $\mathbf{p}$, a conditional TV structure is recursively constructed from the structure. The prefix itemset of the conditional TV is itemset $\mathbf{p}\bullet i$ composed of $\mathbf{p}$ and i. A TV structure consists of a prefix tree, a header

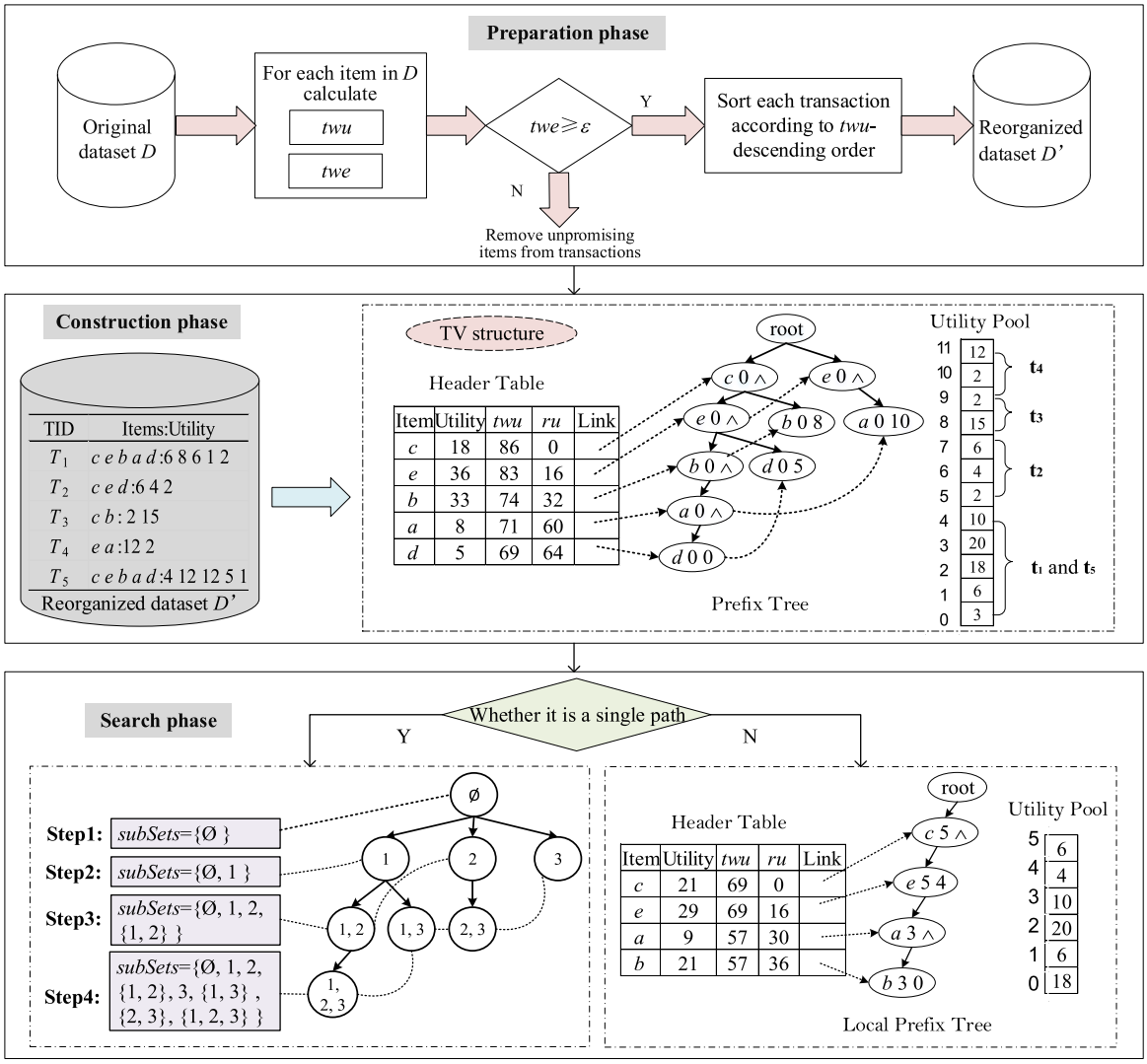

Fig. 1. Framework of the OPHEP-Miner algorithm. This approach involves three key phases: preparation, construction, and search. During the preparation phase, the dataset is reconstructed. The construction phase focuses on establishing a TV structure. In the search phase, there are two distinct paths. If the prefix tree in the TV structure meets the single-path condition, then the bottom-up single-path pattern enumeration method is applied to produce HEPs. Otherwise, HEPs are discovered recursively within the TV structures.

Table 3
Transaction weighted utility of items.

Item	*a*	*b*	*c*	*d*	*e*
Transaction Weighted Utility (*twu* (i_j))	71	74	86	69	83

table, and a utility pool. A prefix tree, denoted as *TV.PT*, is used to store the pattern information of transactions. To build the prefix tree, each transaction is arranged in *twu*-descending order. Furthermore, items with *twe* values below the given efficiency threshold are excluded. Each branch represents a revised transaction.

A header table, denoted as *TV.HT*, is used to store the utility values and links about items in the prefix tree. It consists of five elements: item, utility, *twu*, *ru*, and link.

- Item is used in conjunction with a prefix pattern to generate a larger pattern **p•*i***.
- Utility denotes the utility of **p•*i*** in a dataset.
- *twu* represents the *twu* value of **p•*i*** in a dataset.
- *ru* denotes the *ru* value of **p•*i*** in a dataset.
- Link is utilized to determine the location of **p•*i*** in the prefix tree.

A utility pool, denoted as *TV.UP*, is used to store the utility values in a branch in the prefix tree.

Each node in this tree comprises three elements: item, prefix utility, and utility pool index.

- Item is the item name.
- Prefix utility represents the utility value linked to a prefix pattern. When constructing the initial prefix tree, all prefix utilities are initialized to zero. These values are updated during the recursive mining process.
- Utility pool index denotes the position of the item utility within the utility pool. This index can be used to obtain the utility value. If the utility pool index of an item is *l*, then the utility value of this item is *TV.UP* [*l*]. Notably, only the index of the last item is stored for each transaction because other item indexes can be inferred.

Example 9. In this example, we have $\varepsilon = 0.5$. To construct the TV structure, $\mathbf{t}_1$-$\mathbf{t}_5$ are inserted sequentially. The initial TV structure is an

empty prefix tree. Taking inserting $\mathbf{t}_1 = \{a,b,c,d,e\}$ as an example, according to Table 3 the order is $c \prec e \prec b \prec a \prec d$. Thus, we obtain the prefix tree shown in Fig. 2(a). It can be found that $u_t(c,\mathbf{t}_1) = 6$ since eu(c) = 2 and iu(c,$\mathbf{t}_1$) = 3. According to Example 5, tu($\mathbf{t}_1$) = 23. Thus, the value of twu is 23. According to Definition 7, ru(c) = 0. Hence, we obtain the header table and the utility pool shown in Fig. 2(a). Similarly, after inserting $\mathbf{t}_2 = \{c,d,e\}$, the prefix tree, the header table, and the utility pool are shown in Fig. 2(b). Moreover, after inserting $\mathbf{t}_3$ and $\mathbf{t}_4$, the results are shown in Fig. 2(c). Finally, we insert $\mathbf{t}_5$, and the results are shown in Fig. 3. Note that $\mathbf{t}_1$ and $\mathbf{t}_5$ have the same items and can therefore be merged. During the merge process, the values of the utility pool need to be updated.

4.1.2. Pruning strategies on the TV structure

Pruning strategies designed for the TV structure are introduced as follows.

Definition 9. (Remaining efficiency) The remaining efficiency of pattern $\mathbf{p}$ in transaction $\mathbf{t}_f$ and dataset D are defined as $re_t(\mathbf{p},\mathbf{t}_f) = \frac{\sum_{j\in \mathbf{t}_f \wedge \mathbf{p}\prec j} u_t(j,\mathbf{t}_f)}{inv_d(\mathbf{p})}$ and $re_d(\mathbf{p}) = \frac{ru_d(\mathbf{p})}{inv_d(\mathbf{p})}$, respectively. The order $\prec$ refers to the ascending order of twu.

Example 10. According to Tables 1 and 2, $re_t(\{b,c\},\mathbf{t}_1) = \frac{ru_t(\{b,c\},\mathbf{t}_1)}{inv_d(\{b,c\})} = \frac{10}{78} \approx 0.13$ and $re_d(\{b,c)\} = \frac{ru_d(\{b,c\})}{inv_d(\{b,c\})} = \frac{23}{78} \approx 0.29$.

Definition 10. (Remaining minimum investment) The remaining minimum investment of pattern $\mathbf{p}$ in a TV is defined as $rmi(\mathbf{p}) = \min\{inv_d(i) | \mathbf{p} \prec i \wedge i \in TV.HT\}$. Note that the order $\prec$ refers to the ascending order of twu.

Example 11. According to Fig. 3, we have $rmi(b) = \min\{inv_d(c), inv_d(e)\} = \min\{45, 72\} = 45$ and $rmi(c) = 0$, since there are no extra items to extend c.

Definition 11. (Tighter efficiency upper bound) The tighter efficiency upper bound of pattern $\mathbf{p}$ in a TV structure is defined as $teub(\mathbf{p}) = \frac{u_d(\mathbf{p})}{inv_d(\mathbf{p})} + \frac{ru_d(\mathbf{p})}{inv_d(\mathbf{p})+rmi(\mathbf{p})}$.

Example 12. According to Tables 1 and 2, $teub(b) = \frac{u_d(b)}{inv_d(b)} + \frac{ru_d(b)}{inv_d(b)+rmi(b)} = \frac{33}{33} + \frac{32}{33+45} = 1.41$.

Theorem 2. *For a pattern $\mathbf{p}$, if $teub(\mathbf{p}) < \varepsilon$, then for any pattern $\mathbf{q} \supseteq \mathbf{p}$, it holds that $e_d(\mathbf{q}) < \varepsilon$.*

The proof is provided in Appendix A.

Pruning Strategy 2. For a pattern $\mathbf{p}$, if $teub(\mathbf{p}) < \varepsilon$, then $\mathbf{p}$ and all its supersets are not HEPs.

Property 1. *For a pattern $\mathbf{p}$, we have $twe(\mathbf{p}) > teub(\mathbf{p}) > e_d(\mathbf{p})$.*

The proof is given in Appendix B. According to Property 1, we can conclude that *teub* is a tighter upper bound than *twe*.

4.2. Single-path pattern enumeration method

Our proposed algorithm utilizes the TV structure, inspired by the Hamm algorithm (Qu et al., 2023). Hamm employs a binary tree search strategy to extract HUPs in a top-down manner along a single path. However, this approach is inefficient when applied to HEPM. To illustrate this point, consider Table 1 where the prefix itemset is {d, b}. In this case, the local prefix tree includes nodes {e}, {c}, and {a}. The pruning method employed by the Hamm algorithm in its binary tree search strategy involves pruning all subsets of {d, b, e, c, a} if the utility value falls below a specified threshold. However, this approach is not suitable for HEPM. For instance, we know that $u_d(\{d, b\}) = 21$, $u_d(\{e, c, a\}) = 99$, $u_d(\{c, a\}) = 58$, $inv_d(e) = 70$, $inv_d(a) = 8$, $inv_d(c) = 45$, $inv_d(d, b) = 53$, $e_d(\{d, b, e, c, a\}) \approx 0.682$ and $e_d(\{d, b, c, a\}) \approx 0.745$. If $\varepsilon = 0.7$, then {d, b, e, c, a} is not an HEP. However, its subset {d, b, c, a} is an HEP. Therefore, the entire binary tree search strategy is ineffective for HEPM. Moreover, it is the most critical technique used by the Hamm algorithm to improve its efficiency. To solve this problem, we introduce a novel bottom-up approach for enumerating patterns along a single path. Additionally, we present an innovative pruning technique specifically for single-path trees to constrain the search space.

In our search strategy, a single path implies only one branch in a tree and requires only the last node to possess an index in the utility pool within this branch. This condition ensures that items contained in transactions are entirely identical. For a tree with only a single path, we can consider it as a list. Then, it is only necessary to devise an effective strategy to enumerate all subsets of this list. Taking *itemOrderList* = {e, c, a} as an example, the process of generating all subsets of {e, c, a} is illustrated in Fig. 4.

Step 1: Define a set denoted as *subSets*, whose initial value is an empty set.
Step 2: Merge each item in the *itemOrderList* with the existing patterns in *subSets* one by one. Combining {e} with {Ø} updates *subSets* to {Ø, e}. Note the newly generated patterns in Fig. 4 are in red.
Step 3: Integrate item c with each element in *subSets*, resulting in {Ø, e, c, {e, c}}. Continuing this process, combine a with each element in *subSets*.
Step 4: Consequently, {Ø, e, c, {e, c}, a, {e, a}, {c, a}, {e, c, a}} are obtained, thus generating all the subsets of {e, c, a}.

Given the large number of candidate patterns generated by a single path, an effective pruning strategy is necessary to reduce the search space. Therefore, we propose a novel upper bound for the single-path case in TV structures. To obtain this value, it is first necessary to rank all items along the path in descending order of their investment values. A list named *itemOrderList* is used to store the results. Then, the utility values of all the items in *itemOrderList* are summed. This sum is then added to the utility value of the prefix pattern, and the resulting value is denoted as *maxUtility*. The definition of the single-path efficiency upper

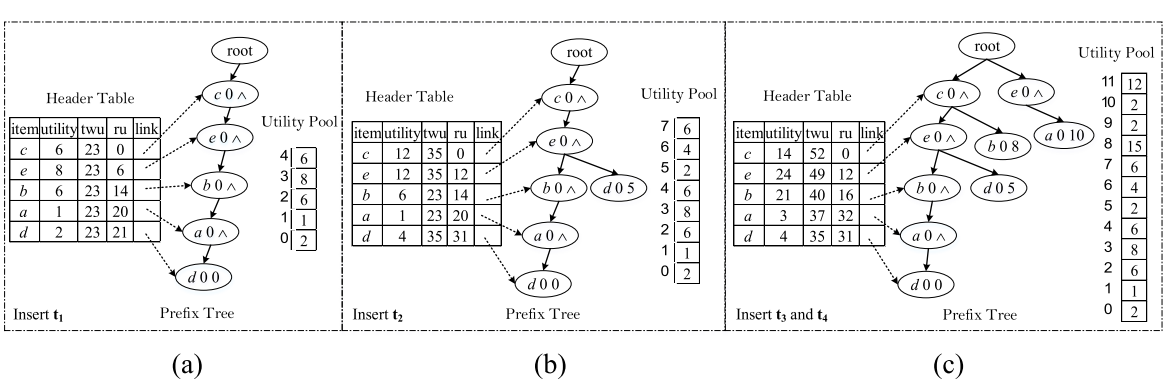

Fig. 2. Construction of the TV structure (a) after inserting $\mathbf{t}_1$, (b) after inserting $\mathbf{t}_2$, (c) after inserting $\mathbf{t}_3$ and $\mathbf{t}_4$.

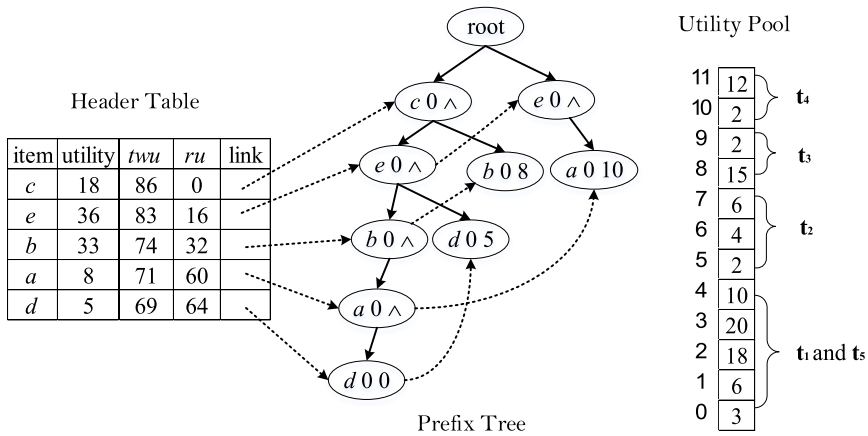

Fig. 3. The TV structure.

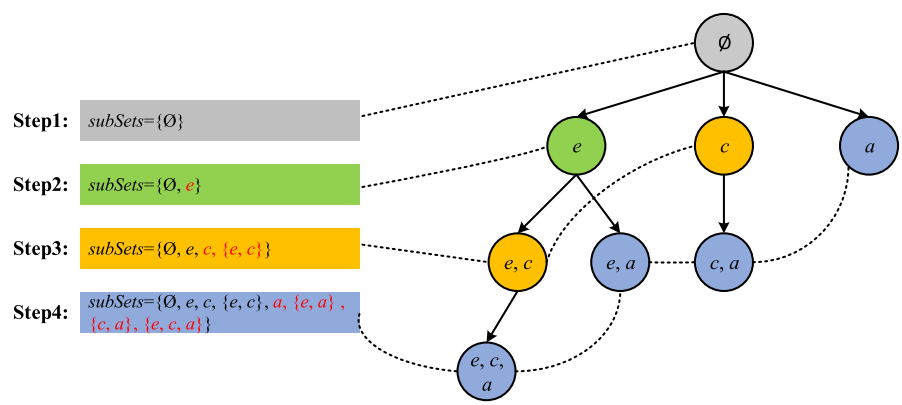

Fig. 4. Illustration of the power set generation process for {*e*, *c*, *a*}.

bound is given below.

Definition 12. **(Single-path efficiency upper bound)** The single-path efficiency upper bound of pattern **p** in a TV structure is defined as $seub(\mathbf{p}) = \frac{maxUtility}{inv_d(\mathbf{p})}$.

Theorem 3. *Given a TV structure with a single path, the items from TV.*HT are stored in itemOrderList and arranged in descending order according to their investment values. *For a pattern* $\mathbf{p} \subseteq itemOrderList$, *if* $seub(\mathbf{p}) < \varepsilon$, *then for any pattern* $\mathbf{q} \supseteq \mathbf{p}$, *it holds that* $e_d(\mathbf{q}) < \varepsilon$.

The proof is given in Appendix C. To enable early pruning of unpromising patterns, we sort the *itemOrderList* by decreasing investment value. For instance, consider a scenario in Table 1 with a pattern $\mathbf{p} = \{a, e\}$, where $inv_d(a) = 8$, $inv_d(e) = 70$, $maxUtility = 89$, and $\varepsilon = 3.5$. In this case, we get that $seub(a) = 11.125$ and $seub(e) \approx 1.271$. If the mining process follows the alphabetical order of items, {*a*, *e*} would be generated as $seub(a) > 3.5$. However, by processing in descending order of investment values, item *e* is first processed. As $seub(e) \approx 1.271 < 3.5$, no supersets of item *e* could be generated. Therefore, this strategic ordering optimizes the pattern generation process.

Pruning Strategy 3. For a pattern $\mathbf{p} \subseteq itemOrderList$, if $seub(\mathbf{p}) < \varepsilon$, then **p** and all its supersets are not HEPs.

For convenient reference, Table 4 provides a summary and explanation of the principal mathematical symbols used in this paper.

To understand the entire pruning mechanism comprehensively, Algorithm 1 provides the pseudo-code for the single-path optimized search strategy. The main steps are as follows.

Table 4
Summary of symbols and their explanations.

Definition	Symbol
The utility of **p** in transaction $\mathbf{t}_f$	$u_t(\mathbf{p}, \mathbf{t}_f)$
The utility of **p** in dataset *D*	$u_d(\mathbf{p})$
The investment of item *j* in transaction $\mathbf{t}_f$	$inv_t(j, \mathbf{t}_f)$
The investment of item *j* in dataset *D*	$inv_d(j)$
The investment of pattern **p** in dataset *D*	$inv_d(\mathbf{p})$
The efficiency of pattern **p** in transaction $\mathbf{t}_f$	$e_t(\mathbf{p}, \mathbf{t}_f)$
The efficiency of pattern **p** in dataset *D*	$e_d(\mathbf{p})$
The transaction utility of transaction $\mathbf{t}_f$	$tu(\mathbf{t}_f)$
The transaction weighted utility of pattern **p**	$twu(\mathbf{p})$
The remaining utility of pattern **p** in transaction $\mathbf{t}_f$	$ru_t(\mathbf{p}, \mathbf{t}_f)$
The remaining utility of pattern ***p*** in dataset *D*	$ru_d(\mathbf{p})$
The transaction-weighted efficiency of pattern **p** in dataset *D*	$twe(\mathbf{p})$
The remaining efficiency of pattern **p** in transaction $\mathbf{t}_f$	$re_t(\mathbf{p}, \mathbf{t}_f)$
The remaining efficiency of pattern **p** in dataset *D*	$re_d(\mathbf{p})$
The remaining minimum investment of pattern **p** in a TV	$rmi(\mathbf{p})$
The tighter efficiency upper bound of pattern **p** in a TV structure	$teub(\mathbf{p})$
The single-path efficiency upper bound of pattern **p** in a TV structure	$seub(\mathbf{p})$

Step 1: In the context of a TV structure comprising only a single path, *LN* represents the leaf node of the prefix tree within the TV structure (Line 1). *maxUtility* is used to calculate the value of *seub*, initially set to the utility value of the prefix pattern (Line 2). *curIndex* indicates the index of the utility pool of the last node in the path (Line 3). *itemOrderList* contains items from all nodes in the path, initially empty (Line 4).

Step 2: To determine the values of these variables, iterate through the header table in reverse order (Lines 5–9).
Step 3: Arrange the items in *itemOrderList* based on their investment values in descending order (Line 10).
Step 4: Following the enumeration method described above, candidate patterns are sequentially generated. If the efficiency of a pattern **p•X** is less than ε, **p•X** is not considered an HEP. Additionally, if the *seub* value of **p•X** falls below ε, none of its supersets are HEPs. Consequently, **X** will not be added to *subSets*, and no superset of **p•X** will be generated (Lines 11–26).

Algorithm 1. SinglePathMine(*TV*, ε, **p**, preInv)

Input: *TV*, a TV structure; ε, the given efficiency threshold; **p**, a prefix pattern; *preInv*, the investment of pattern **p**.
Output: The set of HEPs contained in TV.PT.
//*LN.preUtil*, the prefix utility of the node *LN*; LN.*index*, the index of the utility pool associated with the node LN.

1. LN is the leaf node of TV.PT.
2. *maxUtility* ← LN.*preUtil*;
3. *curIndex* ← LN.*index*;
4. *itemOrderList* ← Ø;
5. **for each** item i in TV.HT from back to front
6. maxUtility ← maxUtility +TV.*UP* [*curIndex*];
7. curIndex ← curIndex +1;
8. itemOrderList ← itemOrderList È *i*;
9. **end for**
10. Arrange itemOrderList based on their investments in descending order.
11. subSets ← Ø;//A list used to store patterns.
12. **for each** item i in itemOrderList
13. len ← subSets.*size*;
14. **for** j = 1 to len
15. **X** ← *subSets* [*j*] È *i*;
16. Calculate the utility and investment values of pattern **p•X**.
17. **if** $e_d(\mathbf{p}.\mathbf{X}) \geq \varepsilon$ **then**
18. SHEPs ← SHEPs È **p•X**;
19. **end if**
20. $seub \leftarrow \dfrac{maxUtility}{inv_d(\mathbf{X}) + preInv}$
21. **if** $seub(\mathbf{p}.\mathbf{X}) \geq \varepsilon$ **then**//Pruning Strategy 3
22. *subSets* ← subSets È **X**;
23. **end if**
24. **end for**
25. **end for**
26. **return** SHEPs

4.3. Pseudo-code for OPHEP-miner

Algorithm 2 demonstrates the pseudo-code for the main function of OPHEP-Miner. The main steps of OPHEP-Miner are as follows. Steps 1 to 5 represent the preparation phase, Step 6 is the construction phase, and Step 8 is the search phase.

Step 1: Scan the dataset once to calculate the values of *twu* and *twe* (Line 1).
Step 2: Another scan of the dataset is conducted to eliminate items with efficiencies below the threshold ε (Line 3).
Step 3: Each transaction is reorganized in descending order based on their *twu* values, generating a new transaction $\mathbf{t}_f'$ (Line 4).
Step 4: The transaction-weighted utility of $\mathbf{t}_f'$ is recalculated and represented as *ntu* (Line 5).
Step 5: $\mathbf{t}_f'$ is then inserted into an empty tree, and the corresponding header table and utility pool are established (Line 6).
Step 6: The search process iterates recursively until all the HEPs are identified (Line 8).

Algorithm 2. OPHEP-Miner(*D*, ε)

Input: *D*, a transaction dataset; ε, the given efficiency threshold
Output: The complete set of HEPs.

1. Calculate the twu and twe values of all items in D.
2. **for each** transaction $\mathbf{t}_f$ in D
3. Remove items from $\mathbf{t}_f$ with twe values no greater than ε.//Pruning Strategy 1
4. A new transaction $\mathbf{t}_f'$ is created by sorting transaction $\mathbf{t}_f$ based on the *twu*-descending order.
5. $ntu \leftarrow tu(\mathbf{t}_f')$;
6. Insert ($\mathbf{t}_f'$, ntu);
7. **end for**
8. Search (*TV*, ε, Ø, 0);
9. **return** HEPs

In the construction phase, a TV structure is employed to store transactions for HEPM. Algorithm 3 shows how to build a TV structure.

Step 1: Initially, items are inserted into the tree according to *twu*-descending order. If the item of the child node of the current node is not *i*, then a new node *Ni* should be generated (Line 3). Node *Ni* comprises three elements as *i*, *prefUtil* and *index*, where *preUtil* represents the utility of the prefix pattern, and *index* denotes the index within the utility pool. Next, the child node of *currentNode* points to *Ni*, and the parent node of *Ni* points to *currentNode* (Line 4). Furthermore, the current node points to *Ni* (Line 5).
Step 2: Simultaneously, upon inserting a node into *TV.PT*, the header table *TV.HT* is updated. If item *i* is not contained in *TV.HT*, then $\langle i, u_t(i,\mathbf{t}), tu, ru_t(i,\mathbf{t})\rangle$ is inserted into *TV.HT*. Otherwise, the relevant elements in *TV.HT* are updated (Lines 6–10). On the other hand, if the item of the child node of the current node is *i*, then the corresponding elements in *TV.HT* are updated, and the current node points to its child node. (Lines 11–13).
Step 3: A utility pool is established. If the transaction is unique, then all the utility values of items in *T* are inserted into *TV.PU*, and only the index of the last item of *T* is provided (Lines 16–18). Otherwise, the utility values in the utility pool are updated (Lines 19–21).

Algorithm 3. Insert(**t**, tu)

Input: **t**, a transaction; *tu*, the transaction utility of **t**.
Output: A TV structure after insertion of transaction **t**.
//*TV.PT*, the prefix tree structure of a TV structure; TV.*HT*, a header table of a TV structure; TV.*UP*, a utility pool of a TV structure; *currentNode*, the current node.

1. *currentNode* ← **t**.*root*;
2. **for each** item i in **t** from front to back
3. **if** currentNode.child.item ≠ i **then**
4. Insert a new Node Ni into TV.PT.
5. currentNode ← *Ni*;
6. **if** i is not in TV.HT **then**
7. $\langle i, u_t(i,\mathbf{t}), tu, ru_t(i,\mathbf{t})\rangle$ is inserted into TV.HT.
8. **else**
9. Update the corresponding elements in TV.HT.
10. **end if**
11. **else**
12. Update the elements about i in TV.HT.
13. currentNode ← currentNode.*child*;
14. **end if**
15. **end for**
16. **if** $\nexists \mathbf{t} = \mathbf{t}'$, and $\mathbf{t}'$ has been inserted into TV.PT **then**
17. Insert the utility of each item in **t** to TV.*UP*;
18. Only the last item in **t** is assigned the index of TV.*UP*;
19. **else**
20. Update the utility value in TV.*UP*;
21. **end if**
22. **return** *TV*;

Algorithm 4 describes the pattern search procedure of the OPHEP-Miner algorithm. The main steps in the search phase are as follows.

Step 1: The *rmi* value of each item is computed for Pruning Strategy 2 (Lines 1–8).
Step 2: Larger patterns are generated based on items in *TV.HT* in a reverse order, corresponding to the ascending order of *twu* values. If the efficiency of a pattern $\mathbf{p}\bullet i$ exceeds or equals to ε, then $\mathbf{p}\bullet i$ is identified as a HEP (Lines 9–14).
Step 3: The *teub* value of pattern $\mathbf{p}\bullet i$ is evaluated (Line 15). If this value falls below ε, then all supersets of $\mathbf{p}\bullet i$ are not HEPs and are pruned by strategy 2.
Step 4: If the *teub* value is no less than ε, then a new TV structure, denoted as *nTV*, is created, comprising a local prefix tree, a header table, and a utility pool. Moreover, the value of *newPreInvest* is updated (Lines 16–18).
Step 5: If the local prefix tree exhibits a single path where solely the last node has a utility pool index, then the single-path optimized search strategy is utilized to extract HEPs (Line 20). Otherwise, the Search function is executed recursively (Line 22). Once all the patterns prefixed with $\mathbf{p}\bullet i$ are generated, the node associated with item i can be released, and then the corresponding pointers are adjusted (Lines 25–26).

Algorithm 4. Search(*TV*, ε, **p**, preInv)

Input: *TV*, a TV structure; ε, the given efficiency threshold; **p**, a prefix pattern; *preInv*, the investment of pattern **p**.
Output: The complete set of HEPs.

1. *minInvest* ← Integer.*Max_Value*;
2. **for each** item i in TV.HT from front to back
3. preItem is the item before i in TV.HT.
4. **if** minInvest > inv_d (preItem)
5. minInvest ← inv_d (preItem);
6. **end if**
7. rmi(i) ← *minInvest*;//Used to calculate teub.
8. **end for**
9. **for each** item i in TV.HT from back to front
10. $\mathbf{p}\bullet i \leftarrow \mathbf{p}$ È i;
11. $e_d(\mathbf{p}.i) \leftarrow \dfrac{u_d(\mathbf{p}.i)}{preInv + inv_d(i)}$;
12. **if** $e_d(\mathbf{p}.i) \geq \varepsilon$ **then**
13. HEPs ← HEPs È $\mathbf{p}\bullet i$;
14. **end if**
15. $teub(\mathbf{p}.i) \leftarrow \dfrac{u_d(\mathbf{p}.i)}{preInv + inv_d(i)} + \dfrac{ru_d(\mathbf{p}.i)}{preInv + inv_d(i) + rmi(i)}$;
16. **if** $teub(\mathbf{p}.i) \geq \varepsilon$ **then**//Pruning Strategy 2
17. Construct the TV structure nTV with a local prefix tree, a header table and a utility pool.
18. newPreInvest ← $preInv + inv_d$(i);
19. **if** the local tree has single path **then**
20. SinglePathMine (*nTV*, ε, $\mathbf{p}\bullet i$, newPreInvest);
21. **else**
22. Search (*nTV*, ε, $\mathbf{p}\bullet i$, newPreInvest);
23. **end if**
24. **end if**
25. Delete the node Ni associated with item i in TV.PT and modify the pointer information.
26. Modify the index of TV.PU of the parent node of Ni.
27. **end for**
28. **return** HEPs

For clarity, an illustrative example of the OPHEP-Miner algorithm is provided in Appendix D.

4.4. Complexity analysis

The time complexity of OPHEP-Miner is $O(m \times n \times t) + O(m \times n \times p)$, where m represents the distinct transactions, n is the distinct items, and t refers to the number of patterns in the search space, and p denotes the maximum number of child nodes of a node in a prefix tree. Algorithm 3 illustrates how to insert a transaction into a TV structure. The complexity of constructing a TV structure is $O(m \times n \times p)$. Additionally, checking if an item is a child of the currentNode takes $O(p)$ time. Algorithms 1 and 4 display the search process for mining the complete HEPs. Let t be the number of patterns in the search space. The search process requires $O(m \times n \times t)$. Algorithm 1 implements a single-path optimization strategy. In the worst-case scenario, assuming the single-path length is l, it will generate 2^l candidate patterns, resulting in a time complexity of $O(2^l + l \times \log l)$. However, in the best-case scenario, no candidate patterns are generated, and the time complexity is $O(l \times \log l)$, primarily due to the time required for sorting the *itemOrderList*. Overall, since 2^l is significantly less than t, the time complexity of the search process is $O(m \times n \times t)$.

5. Experimental analysis

To assess the effectiveness of our proposed OPHEP-Miner algorithm, the following questions need to be analyzed.

Q1. What is the time and space performance of OPHEP-Miner?
Q2. Three pruning strategies are utilized to limit the search space, how do they perform in terms of pruning effectiveness?
Q3. Is the OPHEP-Miner algorithm scalable?
Q4. OPHEP-Miner is based on the TV structure. Compared with other algorithms that also use the TV structure, what are the advantages of the proposed algorithm?
Q5. What is the effectiveness of Pruning Strategy 3 in optimizing performance for single-path scenarios within a prefix tree?

For Q1, we compare OPHEP-Miner with state-of-the-art algorithms: HEPMiner (Zhang et al., 2023b), MHEI (Huynh et al., 2024), and EHEPM (Yildirim, 2025). The results of the time and space performance analyses are presented in Sections 5.2 and 5.3, respectively. Regarding Q2, the number of candidate patterns is compared to assess the pruning effectiveness of the various algorithms in Section 5.4. In relation to Q3, Section 5.5 provides an analysis of the scalability performance of different approaches. For Q4, we have developed a comparative algorithm, HAMM*, based on the state-of-the-art TV structured algorithm Hamm. Since Hamm was designed for mining high utility patterns, we have applied the pruning strategies 1 and 2 proposed in this paper to replace the pruning strategies in Hamm, aiming to achieve HEPM. In response to Q5, Pruning Strategy 3 is a novel pruning technique, we compare it with the method that does not employ this strategy and confirm its effectiveness. All five algorithms were implemented in Java for our experiments. To comprehensively validate our proposed algorithm, we utilized eight datasets for comparative analysis, with detailed dataset characteristics in Table 5. All datasets were obtained from the SPMF library (Fournier-Viger et al., 2016). The experiments were executed on a computer featuring a 3.50-GHz AMD Ryzen 9 3950 16-Core processor with 128 GB of RAM, running on a 64-bit Microsoft Windows 10 operating system. All datasets can be downloaded from https://www.philippe-fournier-viger.com/spmf/index.php?Link=datasets.php.

In Table 5, the dataset density is calculated as the average transaction length divided by the number of distinct items. To generate

Table 5
The characteristics of different datasets.

Datasets	Transactions	Distinct items	Average transaction length	Density (%)
mushroom	8124	119	23	19.33
chess	3196	75	37	49.33
chainstore	1,112,949	46086	7.23	0.02
crimes	2,662,309	35	1.795	5.13
connect	67,557	129	43	33.33
RecordLink	574913	29	10	34.48
t25i10d10k	9976	929	24.77	2.67
c20d10k	10000	192	20	10.42

investment values, we employed the open-source code of the HEPMiner algorithm available on GitHub. Additionally, external utility values followed a Gaussian normal distribution for the RecordLink, t25i10d10k and c20d10k datasets. Internal utility values for items within transactions were randomly assigned between 1 and 100. The evaluation of the five algorithms across eight datasets is based on four crucial metrics: time efficiency, space utilization, candidate pattern count, and scalability.

5.1. Baseline algorithms

The existing high-efficiency pattern mining algorithms are HEPMiner (Zhang et al., 2023b), MHEI (Huynh et al., 2024), and EHEPM (Yildirim, 2025). To validate the effectiveness of the proposed OPHEP-Miner algorithm, it was compared with these competitive algorithms. Moreover, we designed an algorithm named HAMM* based on Hamm (Qu et al., 2023) for comparison.

1. HEPMiner: The utility list framework is widely used for addressing tasks such as low-cost pattern mining and utility-driven pattern

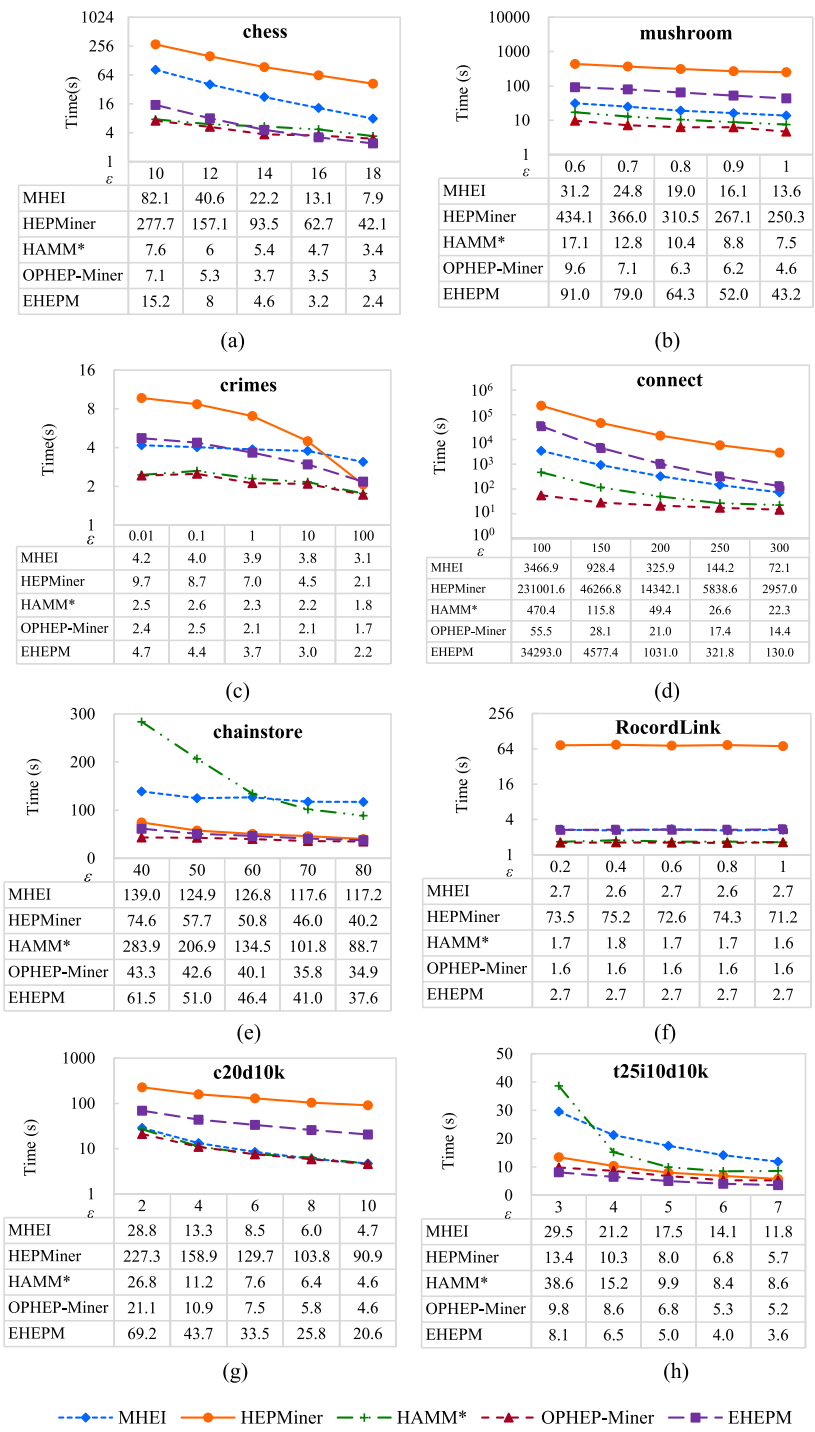

Fig. 5. Runtime for different datasets: (a) chess, (b) mushroom, (c) crimes, (d) connect, (e) chainstore, (f) RecordLink, (g) c20d10k, and (h) t25i10d10k.

mining. HEPMiner achieves high efficiency pattern mining by utilizing the utility list structure and integrating four pruning strategies.
2. MHEI: MHEI, an extension of the EFIM algorithm, uses projected datasets to improve mining performance and employs specified upper bounds for the sub-tree and local efficiency to reduce the number of candidate patterns.
3. EHEPM: Similar to HEPMiner, EHEPM employs a utility list structure but incorporates additional advanced pruning techniques.
4. HAMM*: To validate our single-path pattern enumeration strategy and Pruning Strategy 3, we have also developed a comparative algorithm based on the TV structure for comparison.

5.2. Time efficiency

We evaluated the performance of OPHEP-Miner by comparing it with four other competitive algorithms on eight diverse datasets. To analyze the runtime, memory usage, and scalability of these algorithms, we set efficiency threshold parameters based on the characteristics of each dataset. The specific settings are as follows.

- For the chess dataset, the efficiency threshold was set to 10–18.
- For the mushroom dataset, the efficiency threshold was set to 0.6–1.
- For the crimes dataset, the efficiency threshold was set to 0.01–100.
- For the connect dataset, the efficiency threshold was set to 100–300.
- For the chainstore dataset, the efficiency threshold was set to 40–80.

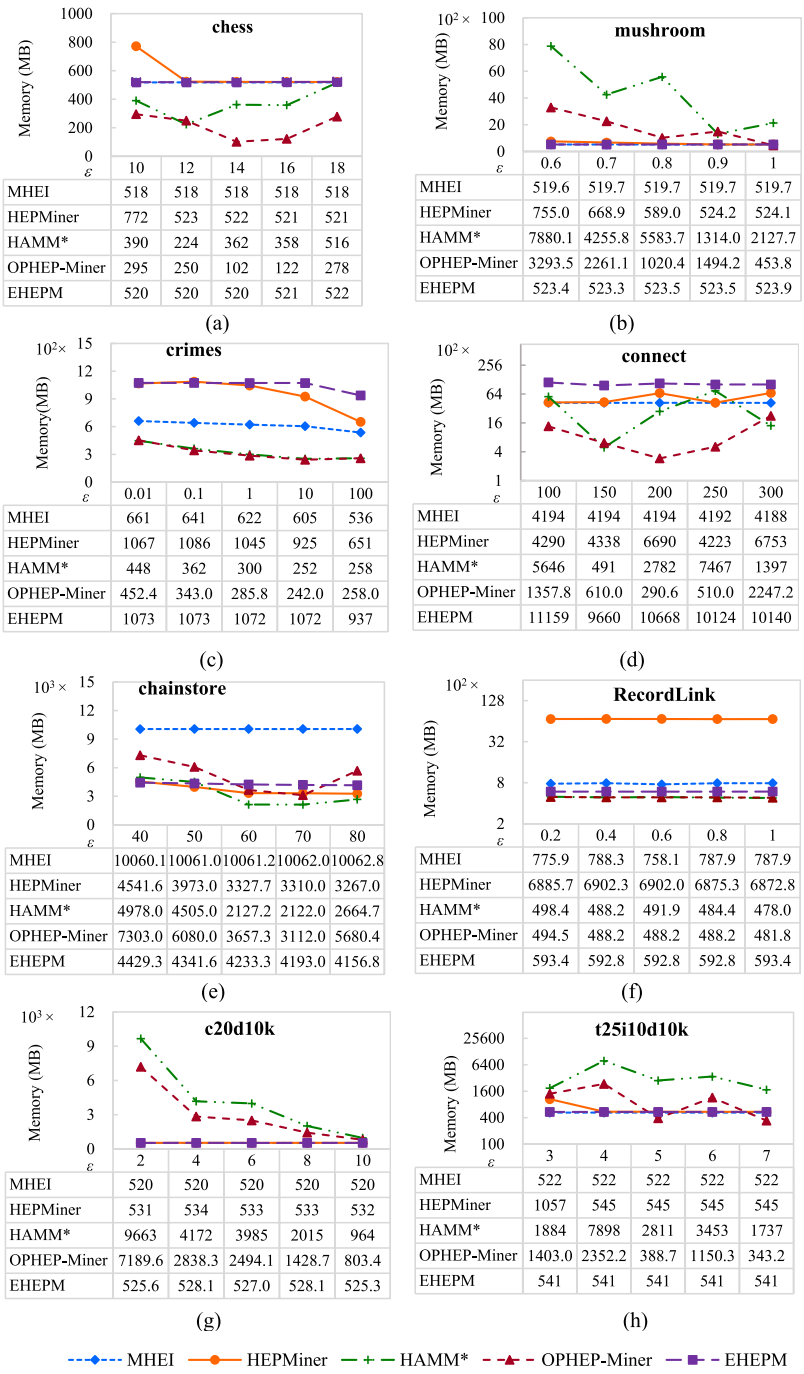

Fig. 6. Space utilization for different datasets: (a) chess, (b) mushroom, (c) crimes, (d) connect, (e) chainstore, (f) RecordLink, (g) c20d10k, and (h) t25i10d10k.

- For the RecordLink dataset, the efficiency threshold was set to 0.2–1.
- For the c20d10k dataset, the efficiency threshold was set to 2–10.
- For the t25i10d10k dataset, the efficiency threshold was set to 3–7.

The following observations are derived from the experimental results.

Fig. 5 demonstrates the runtime performance of five algorithms across eight diverse datasets. The trend clearly indicates a reduction in runtime as the efficiency threshold ε increases. This is because a higher threshold indicates that patterns with higher utility and lower cost meet the criteria. However, as the length of a pattern expands, its construction cost typically rises, while utility does not necessarily grow in proportion, thereby diminishing overall efficiency. From Fig. 5, it can also be seen that the proposed algorithm consistently outperforms other comparative algorithms in most scenarios. For instance, on the connect dataset with ε set at 100, the OPHEP-Miner algorithm is significantly faster than MHEI,

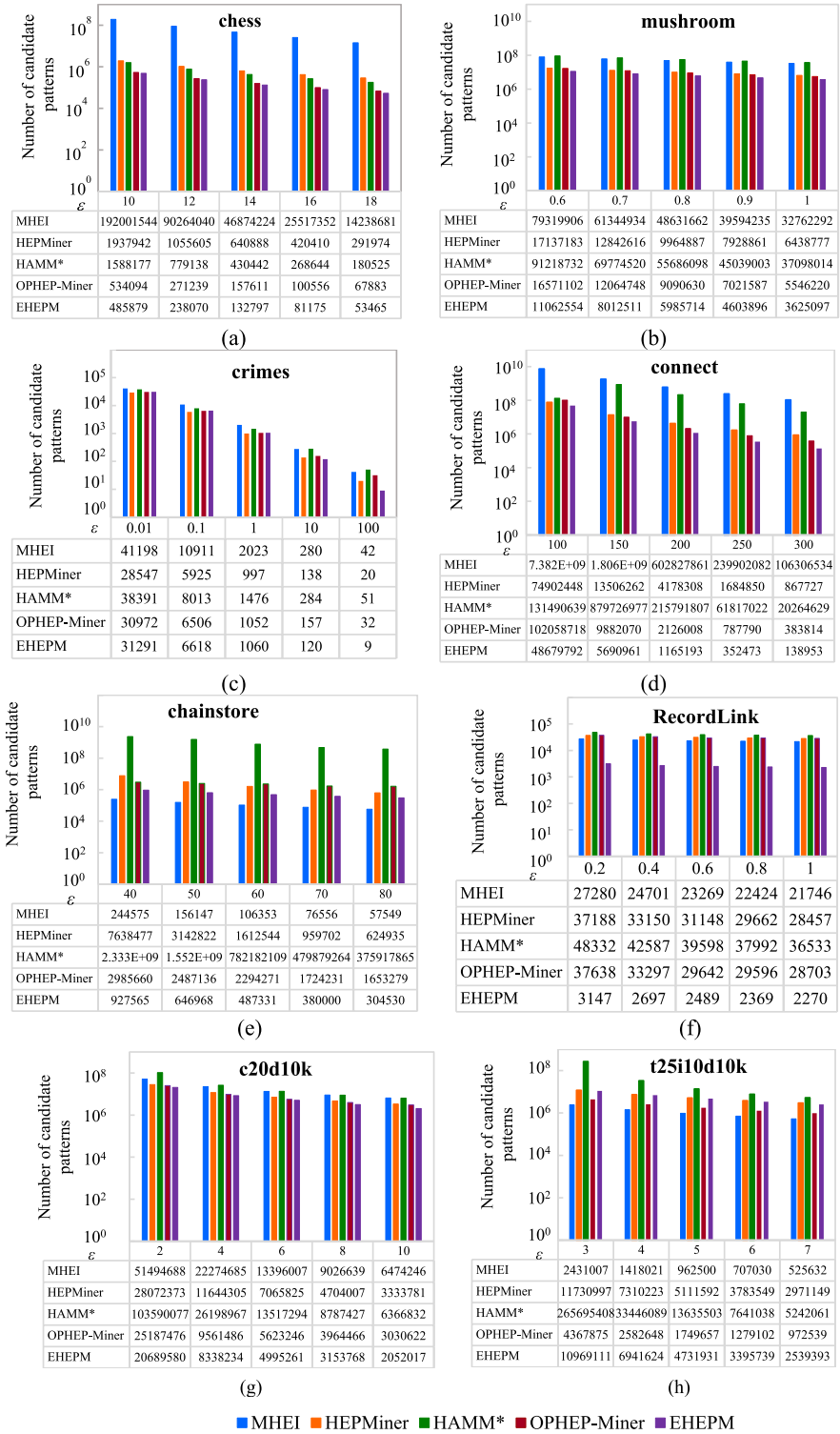

Fig. 7. Number of candidate patterns for different datasets: (a) chess, (b) mushroom, (c) crimes, (d) connect, (e) chainstore, (f) RecordLink, (g) c20d10k, and (h) t25i10d10k.

HEPMiner, HAMM*, and EHEPM, showcasing its efficiency. This superiority can be attributed to the utilization of the utility pool in the TV structure to enhance pattern efficiency. Moreover, the introduction of a single-path optimized search strategy further facilitates the mining process. For the t25i10d10k and chainstore datasets, the HEPMiner and EHEPM algorithms exhibit remarkable performance. This is due to the sparse nature of these datasets. Utility list-based approaches consistently excel on sparse datasets because they generate fewer utility lists. Additionally, Fig. 5 highlights that OPHEP-Miner surpasses HAMM*. For example, on the chainstore dataset with ε set to 40, the OPHEP-Miner algorithm operates significantly faster than HAMM*, mainly because HAMM* lacks an efficient pruning strategy to discard unpromising patterns during the binary search process. This deficiency makes the TV structure-based algorithm less efficient in comparison.

5.3. Space utilization

To evaluate the space occupancy performance of our algorithm, the comparisons of the space occupancy on eight datasets are shown in Fig. 6. For the c20d10k, crimes, and mushroom datasets, it is obvious to see that memory usage decreases as the threshold value increases. This is because a higher threshold value leads to the generation of fewer HEPs. From Fig. 6, we also observe that the OPHEP-Miner algorithm achieves better performance in most cases. For example, on the connect dataset with ε set at 250, the MHEI, HEPMiner, HAMM*, and EHEPM algorithms take up 8.2, 8.2, 14.6, and 19.8 times more space than OPHEP-Miner, respectively. This is because OPHEP-Miner generates the fewest candidate patterns in most cases, and transactions are highly compressed onto the transaction tree. Furthermore, the EHEPM algorithm exhibits similar memory usage to the HEPMiner algorithm, consuming more space on dense datasets but less on sparse datasets. This is primarily because both algorithms utilize utility-list structures, with the EHEPM algorithm employing more powerful pruning strategies. For the mushroom, c20d10k, and t25i10d10k datasets, HAMM* takes up the most memory compared with other approaches. It is caused by the large number of candidate patterns generated. Even though both the HAMM* and OPHEP-Miner algorithms utilize the TV structure for mining, HAMM* lacks an effective pruning strategy in the binary tree search process.

5.4. Number of candidate patterns

To verify the pruning performance of our algorithm, the comparisons of the number of candidate patterns on eight datasets are shown in Fig. 7. The OPHEP-Miner algorithm demonstrates excellent performance on the majority of datasets. This is because OPHEP-Miner recursively generates candidate patterns through prefix trees, which can greatly avoid generating undesirable patterns. Besides, a novel and effective pruning strategy is specially designed to further reduce the search space. However, EHEPM shows superior performance on the RecordLink dataset. This is due to the utilization of the four powerful pruning strategies to limit the search space. It can also be seen in Fig. 7 that the MHEI algorithm performs the best on the chainstore and t25i10d10k datasets. This is due to the introduction of efficiency upper bounds, namely *ssef* and *slef*, which are stricter than the upper bounds used by the other algorithms. We also see that HAMM* has the worst performance on the chainstore, c20d10k, t25i10d10k and RecordLink datasets. This is because HAMM* uses a binary tree search method to generate HEPs, which lacks an effective pruning mechanism to eliminate unpromising candidates. As a result, despite sharing the same TV-based framework as OPHEP-Miner, HAMM* underperforms due to its inability to efficiently reduce the search space.

5.5. Scalability

To verify the scalability of our algorithm, scalability comparisons are conducted on two synthetic datasets by incrementally increasing the number of transactions in terms of time, space, and the size of candidate patterns. Fig. 8(a)–(c) show the results of the scalability run with a threshold of 14 on the t25i10d|x|k dataset. It is obvious to see that the running time of the algorithm grows as the size of the dataset increases. It is reasonable because larger datasets require more time for traversal. In addition, we observe that the OPHEP-Miner algorithm has a lower running time than the other algorithms. Besides, we also see that the advantage becomes increasingly obvious as the amount of data increases. From Fig. 8(b), it can be found that OPHEP-Miner takes up the least amount of memory in most cases. This is because the algorithm produces the least number of candidate patterns. As shown in Fig. 8(c), the OPHEP-Miner algorithm generally produces fewer candidate patterns than other algorithms on the t25i10d|x|k dataset. This is due to a pruning strategy designed specifically for scenarios with only a single path within a prefix tree. Fig. 8(d)–(f) show the results of five different algorithms running on the c20d|x|k dataset with ε set at 10. As can be seen from these figures, the running time, memory consumption, and candidate pattern size of different algorithms exhibit an increasing trend as dataset size increases. In addition, the OPHEP-Miner algorithm has outstanding performance at runtime, memory consumption, and scalability. Notably, as shown in Fig. 8(a), the runtime of the HAMM* algorithm greatly increases when the dataset size reaches 50,000. The main reason for this is the absence of a strong pruning strategy within this algorithm, leading to a significant drop in runtime efficiency and memory efficiency.

5.6. The effectiveness of pruning strategies

To better validate the effectiveness of the proposed Pruning Strategy 3, we conducted a comparative analysis between our proposed OPHEP-Miner algorithm and an alternative version of OPHEP-Miner that excludes the application of Pruning Strategy 3, in terms of time, space, and the number of candidate patterns. The experimental results are detailed in Table 6. It is evident that Pruning Strategy 3 consistently reduces the number of candidate patterns across all datasets, leading to decreased memory usage and enhanced mining efficiency. For instance, Pruning Strategy 3 has significantly decreased the number of candidate patterns by 42, 97, and 5 times on the connect, chainstore, and t25i10d10k datasets, respectively. Especially on the chainstore and connect datasets, OPHEP-Miner has improved time efficiency by five times compared to when Pruning Strategy 3 was not employed. Moreover, we find that Strategy 3 exhibited a slight advantage on the other datasets. This is because Pruning Strategy 3 is only applied when a prefix tree satisfies the single-path condition. As a result, the characteristics of the dataset affect the effectiveness of this pruning strategy. In conclusion, Pruning Strategy 3 is effective in limiting the search space.

5.7. Case study

HEPM considers both investment and utility factors to discover interesting patterns. To provide a clearer demonstration of the application of HEPM, we utilize a movie dataset available at https://www.kaggle.com/code/abhayayare/movie-recommendation-using-svd/input to demonstrate the application of HEPM in movie recommendation systems. In this framework, movie ratings are set as utility values, while movie release dates are regarded as investments. The rating scale ranges from 1 to 5, where higher values indicate better user ratings for the movies. The release dates of movies are continuous values. Movies released after 2020 are assigned an investment value of 1, those released between 2011 and 2020 are assigned a value of 1.1, movies from 2001 to 2010 are assigned a value of 1.2, those between 1991 and 2000 are assigned a value of 1.3, and movies released before 1991 are assigned a value of 1.4. The efficiency value of a movie is calculated as the rating value divided by the timeliness value. Therefore, HEPM can discover movie combinations with high ratings and timeliness.

To better validate the effectiveness of the HEPM algorithm, we

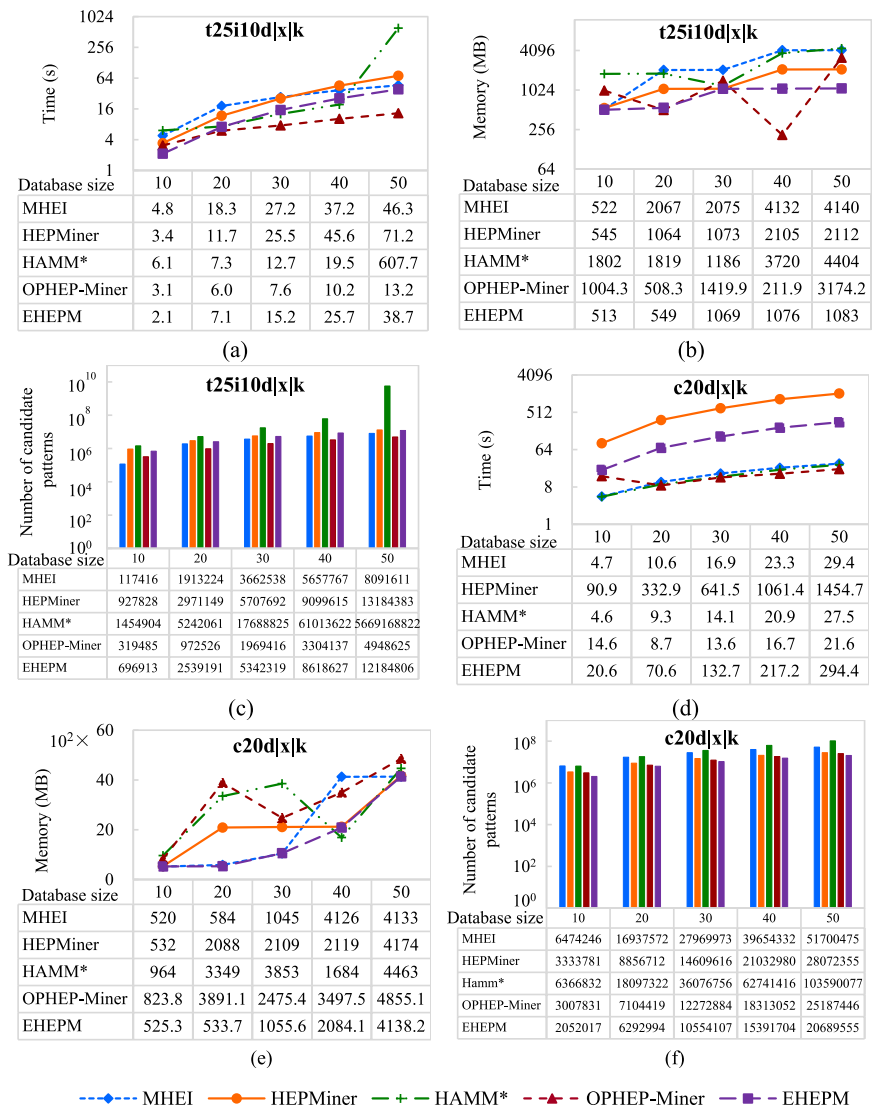

Fig. 8. Scalability for different datasets: (a) Time on t25i10d|x|k; (b) Memory on t25i10d|x|k; (c) Candidates on t25i10d|x|k; (d) Time on c20d|x|k; (e) Memory on c20d|x|k; (f) Candidates on c20d|x|k.

Table 6
Comparison of experimental results for Pruning Strategy 3.

dataset	ε	OPHEP-Miner			OPHEP-Miner-without Pruning Strategy 3		
		Runtime(s)	Memory (MB)	Number of candidates	Runtime (s)	Memory (MB)	Number of candidates
chess	10	7.185	295.72	534094	9.191	434.52	1511335
mushroom	0.6	8.943	3293.5	16570870	13.99	3787.07	90768026
crimes	0.01	2.426	452.415	30970	2.457	452.45	31977
connect	100	55.492	1357.8	102053050	281.988	6853.22	4371797803
chainstore	60	40.09	3657.324	2294260	203.336	7730.63	224831245
RecordLink	0.8	1.629	478.02	29805	1.645	478.04	32138
c20d10k	6	6.672	2430.44	5659105	8.797	2566.89	12963712
t25i10d10k	3	11.405	1402.98	4367875	13.502	4729.16	40250648

compare it against both frequent pattern mining and high-utility pattern mining algorithms for movie recommendations. Table 7 displays the top-5 items derived from three distinct pattern mining algorithms. As the table indicates, the top-five recommended movies differ depending on the algorithm. For example, a movie with ID number 94 does not appear in the mining results of either HUPM or FPM. Although the movie with ID 94 does not have particularly high utility value and support, its release date is after 2020, indicating better timeliness. Therefore, the HEPM mining algorithm prioritizes this movie over the HUPM and FPM mining algorithms. Furthermore, we observed that FPM mined a top-5 list comprising 9 movies, as 8 movies had a support count of 8. Hence, it can be concluded that the HEPM algorithm exhibits better pattern discrimination.

Table 8 displays the top-5 2-itemsets. It is evident that different mining algorithms produce distinct movie combinations. Additionally, the frequent pattern mining algorithm actually generated 54 different

Table 7
Top-5 items by different pattern mining algorithms.

HEPM				HUPM		FPM			
Top-5 items	Utility	Invest	Efficiency	Top-5 items	Utility	Top-5 items	Sup	Top-5 items	Sup
94	25	1.0	25	39	33	39	10	30	8
53	32	1.3	24.6	53	32	92	8	88	8
90	26	1.1	23.6	48	28	48	8	67	8
39	33	1.4	23.5	34	27	49	8		
48	28	1.2	23.3	49	26	77	8		
				90	26	53	8		

Table 8
Top-5 2-itemsets by different pattern mining algorithms.

HEPM				HUPM		FPM					
Top-5 2-itemsets	Utility	Invest	Efficiency	Top-5 2-itemsets	Utility	Top-5 2-itemsets	Sup	Top-5 2-itemsets	Sup	Top-5 2- itemsets	Sup
39 82	29	2.5	11.6	39 82	29	77 39	4	23 10	3	89 96	3
11 74	24	2.3	10.43	6 39	26	6 39	4	32 60	3	31 20	3
79 93	25	2.4	10.41	22 24	26	93 79	4	24 22	3	87 52	3
6 39	26	2.5	10.4	79 93	25	28 77	4	24 92	3	49 65	3
22 24	26	2.5	10.4	30 53	25	14 39	3	82 39	3	4 30	3
						11 74	3	22 41	3	…	3

combinations, but due to space constraints, not all of them are listed. This indicates that HEPM and HUPM have better pattern discrimination. Unlike the top-5 movie lists, combination recommendations can be made based on the results from 2-itemsets. For instance, when a user views movie number 39, movie number 82 can be further recommended.

This example illustrates the application of high-efficiency patterns in real-world scenarios. In the above movie recommendation example, utility is defined as the popularity of a movie, while cost corresponds to the time elapsed since its release. As shown by Tables 7 and 8, HEPM recommends "Building Throughout Among" as the best individual movie, and the pair "Sign Police How"– "Although Safe" is recommended as the best movie combination. In high-efficiency pattern mining, the specific meanings of "utility" and "cost" are context-dependent and should be defined according to the requirements of each application domain. For instance, in travel route recommendation, users tend to prefer routes that are both popular and cost-effective. Hence, utility can be measured by the route's popularity, and cost by its total price. This flexible semantic mapping demonstrates the model's adaptability and practicality across diverse application domains.

6. Conclusions and future work

HEPM is an innovative data mining task that considers both utility and cost metrics to identify patterns that provide more utility at lower costs. To improve the efficiency, we present a one-phase HEPM approach, named OPHEP-Miner, based on TV structures. Because the TV structures are frequently generated during the pattern mining process, we proposed a novel bottom-up enumeration technique for a single path to increase efficiency. This approach effectively avoids the tedious task of constructing pattern trees, thus increasing mining efficiency. Additionally, three pruning strategies have been developed, notably including pattern pre-pruning designed for single-path scenarios. To assess the performance of our proposed algorithm, we compare our proposed algorithm with the current state-of-the-art HEPMiner, MHEI, HAMM*, and EHEPM algorithms. Through experiments on eight different datasets, the results show that our proposed algorithm exhibits superior performance in terms of runtime, space utilization, candidate pattern size, and scalability compared with the state-of-the-art algorithms. Furthermore, in practical applications, the proposed algorithm successfully extracts films with both high ratings and strong timeliness.

However, since the threshold for high-efficiency patterns is determined by a combination of utility and cost, it can be challenging for users to set an appropriate threshold value. To address this issue, a promising direction for future work is to develop a top-k high-efficiency pattern mining approach, which would eliminate the need for users to specify a threshold.

CRediT authorship contribution statement

Xuan Liu: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation. **Genlang Chen:** Writing – review & editing, Project administration, Funding acquisition. **Fangyu Wu:** Writing – review & editing, Funding acquisition, Conceptualization. **Wanli Zuo:** Validation, Formal analysis. **Youxi Wu:** Writing – review & editing, Validation, Supervision, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This study is supported by the Key Technology R&D Program of Ningbo (2024Z259, 2022Z149); Suzhou Science and Technology Development Planning Program (ZXL2023176); Ningbo Philosophy and Social Sciences Planning Project (G25-5-JY23).

Appendix A. Proof of Theorem 2

For a pattern $\mathbf{q}$ that satisfies $\mathbf{p} \subset \mathbf{q}$, we have $e_d(\mathbf{q}) = \frac{u_d(\mathbf{q})}{inv_d(\mathbf{q})}$.

$$\because teub(\mathbf{p}) = \frac{u_d(\mathbf{p})}{inv_d(\mathbf{p})} + \frac{ru_d(\mathbf{p})}{inv_d(\mathbf{p}) + rmi(\mathbf{p})}$$

$$\geq \frac{u_d(\mathbf{p})}{inv_d(\mathbf{p})} + \frac{ru_d(\mathbf{p})}{inv_d(\mathbf{q})}$$

$$> \frac{u_d(\mathbf{p})}{inv_d(\mathbf{q})} + \frac{ru_d(\mathbf{p})}{inv_d(\mathbf{q})} = \frac{u_d(\mathbf{p}) + ru_d(\mathbf{p})}{inv_d(\mathbf{q})}$$

$$\geq \frac{u_d(\mathbf{q})}{inv_d(\mathbf{q})} = e_d(\mathbf{q})$$

$$\therefore e_d(\mathbf{q}) < teub(\mathbf{p})$$

$$\because teub(\mathbf{p}) < \varepsilon$$

$$\therefore e_d(\mathbf{q}) < \varepsilon$$

Next, if $\mathbf{p} = \mathbf{q}$, then prove that $e_d(\mathbf{q}) < \varepsilon$.

$$\because teub(\mathbf{p}) = \frac{u_d(\mathbf{p})}{inv_d(\mathbf{p})} + \frac{ru_d(\mathbf{p})}{inv_d(\mathbf{p}) + rmi(\mathbf{p})} > \frac{u_d(\mathbf{p})}{inv_d(\mathbf{p})} = e_d(\mathbf{p}) = e_d(\mathbf{q})$$

$$\because teub(\mathbf{p}) < \varepsilon$$

$$\therefore e_d(\mathbf{q}) < teub(\mathbf{p}) < \varepsilon$$

Appendix B. Proof of Property 1

Proof: $twe(\mathbf{p}) = \frac{twu(\mathbf{p})}{inv_d(\mathbf{p})} = \frac{\sum_{\mathbf{p} \subseteq \mathbf{t}_f} tu(\mathbf{t}_f)}{inv_d(\mathbf{p})}$

$$\because ru_d(\mathbf{p}) = \sum_{\mathbf{p} \subseteq \mathbf{t}_f \wedge \mathbf{t}_f \in D} \sum_{j \in \mathbf{t}_f \wedge \mathbf{p} \prec j} u_t(j, \mathbf{t}_f)$$

$$\therefore tu(\mathbf{t}_f) \geq u_t(\mathbf{p}, \mathbf{t}_f) + ru_t(\mathbf{p}, \mathbf{t}_f)$$

$$\therefore \frac{\sum_{\mathbf{p} \subseteq \mathbf{t}_f} tu(\mathbf{t}_f)}{inv_d(\mathbf{p})} \geq \frac{u_d(\mathbf{p}) + ru_d(\mathbf{p})}{inv_d(\mathbf{p})}$$

$$> \frac{u_d(\mathbf{p})}{inv_d(\mathbf{p})} + \frac{ru_d(\mathbf{p})}{inv_d(\mathbf{p}) + rmi(\mathbf{p})} = teub(\mathbf{p})$$

$$> \frac{u_d(\mathbf{p})}{inv_d(\mathbf{p})} = e_d(\mathbf{p})$$

Appendix C. Proof of Theorem 3

For a pattern $\mathbf{q}$ that satisfies $\mathbf{p} \subseteq \mathbf{q} \subseteq itemOrderList$, we have $e_d(\mathbf{q}) = \frac{u_d(\mathbf{q})}{inv_d(\mathbf{q})}$.

$$seub(\mathbf{p}) = \frac{maxUtility}{inv_d(\mathbf{p})} = \frac{\sum_{i=0}^{|TV.PT|-1} TV.PT[i] + prefixUtility}{inv_d(\mathbf{p})}$$

where *prefixUtility* refers to the utility of the prefix pattern in *TV.PT*, and this value can be obtained from a node within the single path.

$\because \sum_{i=0}^{|TV.PT|-1} TV.PT[i]$ is the sum of the utility values of all items in *itemOrderList*.

$$\therefore \frac{\sum_{i=0}^{|TV.PT|-1} TV.PT[i] + prefixUtility}{inv_d(\mathbf{p})} \geq \frac{u_d(\mathbf{q})}{inv_d(\mathbf{p})}$$

$$\because inv_d(\mathbf{p}) \leq inv_d(\mathbf{q})$$

$$\therefore seub(\mathbf{p}) \geq \frac{u_d(\mathbf{q})}{inv_d(\mathbf{q})} = e_d(\mathbf{q})$$

$$\because seub(\mathbf{p}) < \varepsilon$$

$\therefore e_d(\mathbf{q}) \leq seub(\mathbf{p}) < \varepsilon$

Appendix D. An illustrative example of OPHEP-Miner

With an efficiency threshold set at 0.5, we demonstrate the step-by-step operation of the OPHEP-Miner algorithm with dataset D shown in Table 1. In preparation phase, we conduct the following six steps.

Step 1: The *twu* and *twe* values are calculated for all the items in D. Tables 3 and 9 show the results for *twu* and *twe*, respectively.
Step 2: Since the efficiency of all the items is higher than 0.5, no item is eliminated by Pruning Strategy 1.

Table 9
Transaction weighted efficiency of items

Item	*a*	*b*	*c*	*d*	*e*
Transaction Weighted Efficiency	8.875	2.24	1.91	3.45	1.15

Steps 3–5: According to the *twu*-descending order, the TV structure depicted in Fig. 2 is constructed.
Step 6: A depth-first recursive search is performed on the TV structure to mine all HEPs.

In the search phase, the following steps are performed.

Step 1: The *rmi* value of each item is calculated, the results are displayed in Table 10.

Table 10
Remaining minimum investment of items

Item	*a*	*b*	*c*	*d*	*e*
Remaining Minimum Investment	33	45	0	8	45

Step 2: We start by considering item d. Based on the index of the utility pool, it is determined that $u_d(d) = 5$ and $e_d(d) = 0.25$. Consequently, d is not an HEP.

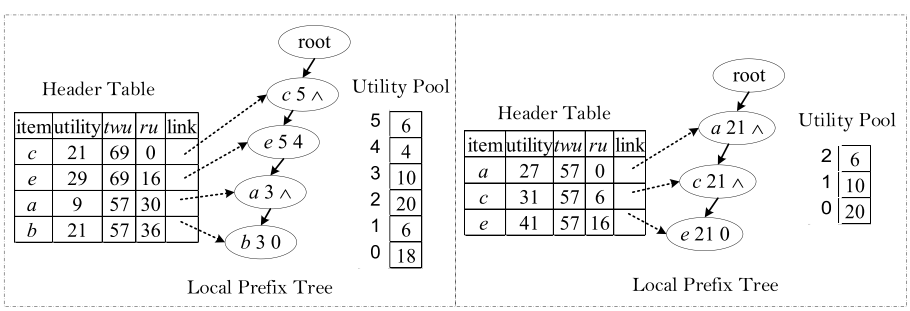

Fig. 9. (a) The local TV structure for {*d*} Fig. 9(b) The local TV structure for {*d*, *b*}

Step 3: We further calculate the *teub* value of d, and $teub(d) = \frac{u_d(d)}{inv_d(d)} + \frac{ru_d(d)}{inv_d(d)+rmi(d)} = \frac{5}{20} + \frac{64}{20+8} \approx 2.54$. Therefore, d should be extended to larger patterns.
Step 4: The prefix pattern is set to d, and a local TV structure shown in Fig. 9(a) is constructed.
Step 5: The Search process is executed. Note that while there is only one path in the local tree, to meet our specified single-path condition, it is essential to ensure that only the last node in the entire path has an index. Hence, the single-path optimization strategy cannot be used.

Then, this search function is executed again. We start by extending d with b to generate the pattern $\{d, b\}$. Because $e_d(\{d,b\}) \approx 0.37$, $\{d, b\}$ is not a HEP. Moreover, $teub(\{d,b\})$ is calculated. The result is greater than 0.5. Subsequently, the local TV structure about $\{d, b\}$ is created, which is displayed in Fig. 9(b). It can be seen that the local prefix tree has only one path, and only the last item has the index of utility pool. Thus, the single-path optimization strategy is employed.
In the single-path search process, the following steps are executed.

Step 1: Because the leaf node of the tree shown in Fig. 9(b) is *e*, we have *LN* refers to *e*. Then, *LN.preUtil* is 21 and assigned to *maxUtility*. We also have the utility pool index of *LN* is 0 (Lines 1–3).
Step 2: Scan the header table from back to front, update the value of *maxUtility* to 57, and create a set *itemOrderList* = {*e*, *c*, *a*}. Note that the prefix pattern is {*d*, *b*}, and the investment is 53.
Step 3: We arrange the items in *itemOrderList* in descending order based on their investments, and the result remains {*e*, *c*, *a*}.
Step 4: We generate all the power sets of the items in *itemOrderList*, and Pruning Strategy 3 is utilized to limit the search space. In order to produce all subsets of {*e*, *c*, *a*} a list *subSets* is created with the initial value of the empty set. Next, iterate through the collection *itemOrderList*, item *e* is first considered. We combine item *e* with each of the patterns in *subSets*. Because *subSets* = {Ø} and the prefix pattern is $\{d,b\}$, a larger pattern $\{d,b,e\}$ is generated. Furthermore, since $e_d(\{d,b,e\}) = \frac{u_d(\{d,b,e\})}{inv_d(\{d,b\})+inv_d(e)} = \frac{41}{53+72} = 0.328$, $\{d,b,e\}$ is not a HEP. Subsequently, we calculate the *seub* value of $\{d,b,e\}$, and $seub(\{d,b,e\}) = \frac{maxUtility}{inv_d(\{d,b\})+inv_d(e)} = \frac{57}{53+72} = 0.456 < \varepsilon$. Thus, the supersets of {*d*, *b*, *e*} cannot be HEPs. Then, Pruning Strategy 3 is applied to eliminate these unpromising itemsets by not adding item *e* to the set *subSets*. Next, $\{d,b,c\}$ is considered. The above steps are repeated until the complete HEPs are mined, and the results are $\{\{a\},\{a,b\},\{a,e\},\{b\},\{b,c\},\{e\}\}$.

Data availability

A link has been included in this manuscript to provide access to the study's data.

References

Agrawal, R., Srikant, R., Year. Fast algorithms for mining association rules. In: Proceedings of the 20th International Conference on Very Large Data Bases,Santiago, Chile, pp.487-499.

Ahmed, C.F., Tanbeer, S.K., Jeong, B.-S., Lee, Y.K., 2009. Efficient tree structures for high utility pattern mining in incremental databases. IEEE Trans. Knowl. Data Eng. 21, 1708–1721.

Chee, C.-H., Jaafar, J., Aziz, I.A., Hasan, M.H., Yeoh, W., 2019. Algorithms for frequent itemset mining: a literature review. Artif. Intell. Rev. 52, 2603–2621.

Dong, X., Gong, Y., Cao, L., 2020. e-RNSP: an efficient method for mining repetition negative sequential patterns. IEEE Trans. Cybern. 50, 2084–2096.

Duong, Q.-H., Fournier-Viger, P., Ramampiaro, H., Nørvåg, K., Dam, T.L., 2018. Efficient high utility itemset mining using buffered utility-lists. Appl. Intell. 48, 1859–1877.

Fournier-Viger, P., Li, J., Lin, J.C.-W., Chi, T.T., Kiran, R.U., 2020a. Mining cost-effective patterns in event logs. Knowl.-Based Syst. 191, 105241.

Fournier-Viger, P., Li, J., Lin, J.C.W., Truong, T., 2020b. Discovering low-cost high utility patterns. Data Sci. Pattern Recognition 4, 50–64.

Fournier-Viger, P., Lin, J.C.-W., Gomariz, A., Gueniche, T., Soltani, A., Deng, Z., Lam, H. T., 2016. The SPMF open-source data mining library version 2. In: Machine Learning and Knowledge Discovery in Databases,Cham, pp. 36–40.

Gan, W., Lin, J.C.-W., Fournier-Viger, P., Chao, H.-C., Philip, S.Y., 2019. HUOPM: High-utility occupancy pattern mining. IEEE Trans. Cybern. 50, 1195–1208.

Gan, W., Lin, J.C.-W., Fournier-Viger, P., Chao, H.-C., Tseng, V.S., Yu, P.S., 2021. A survey of utility-oriented pattern mining. IEEE Trans. Knowl. Data Eng. 33, 1306–1327.

Geng, M., Wu, Y., Li, Y., Liu, J., Fournier-Viger, P., Zhu, X., Wu, X., 2023. RNP-Miner: repetitive nonoverlapping sequential pattern mining. IEEE Trans. Knowl. Data Eng. 1–16.

Han, M., Liu, S., Gao, Z., Mu, D., Li, A., 2024. Mining Top-K constrained cross-level high-utility itemsets over data streams. Knowl. Inf. Syst. 1–40.

He, J., Han, X., Wan, X., Wang, J., 2024. Efficient skyline frequent-utility itemset mining algorithm on massive data. IEEE Trans. Knowl. Data Eng. 1–14.

Ho, V.L., Ho, N., Pedersen, T.B., Papapetrou, P., 2025. Efficient generalized temporal pattern mining in time series using mutual information. IEEE Trans. Knowl. Data Eng. 1–18.

Huynh, B., Tung, N., Nguyen, T.D., Bui, Q.-T., Nguyen, L.T., Yun, U., Vo, B., 2024. An efficient strategy for mining high-efficiency itemsets in quantitative databases. Knowl.-Based Syst., 112035

Iqbal, M., Setiawan, M.N., Irawan, M.I., Khalif, K.M.N.K., Muhammad, N., Aziz, M.K.B. M., 2022. Cardiovascular disease detection from high utility rare rule mining. Artif. Intell. Med. 131, 102347.

Jazayeri, A., Yang, C.C., 2024. Frequent pattern mining in continuous-time temporal networks. IEEE Trans. Pattern Anal. Mach. Intell. 46, 305–321.

Khalil, J., Yan, D., Yuan, L., Han, J., Adhikari, S., Long, C., Zhou, Y., 2024. Fsm-explorer: an interactive tool for frequent subgraph pattern mining from a big graph. In: 2024 IEEE 40th International Conference on Data Engineering (ICDE), pp. 5405–5408.

Kim, H., Yun, U., Baek, Y., Kim, J., Vo, B., Yoon, E., Fujita, H., 2021. Efficient list based mining of high average utility patterns with maximum average pruning strategies. Inf. Sci. 543, 85–105.

Krishnamoorthy, S., 2015. Pruning strategies for mining high utility itemsets. Expert Syst. Appl. 42, 2371–2381.

Lan, G.-C., Hong, T.-P., Tseng, V.S., 2014. An efficient projection-based indexing approach for mining high utility itemsets. Knowl. Inf. Syst. 38, 85–107.

Li, Y., Wang, Z., Liu, J., Guo, L., Fournier-Viger, P., Wu, Y., Wu, X., 2025. Mining repetitive negative sequential patterns with gap constraints. ACM Trans. Knowl. Discov. Data 1–29.

Liu, J., Wang, K., Fung, B.C., 2012. Direct discovery of high utility itemsets without candidate generation. In: Proceedings of the 12 Th International Conference on Data Mining, pp. 984–989.

Liu, M., Qu, J., Year. Mining high utility itemsets without candidate generation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management,Maui, Hawaii, USA, pp.55-64.

Liu, X., Chen, G., Wen, S., Zuo, W., 2024. Effective approaches for mining correlated and low-average-cost patterns. Knowl.-Based Syst. 302, 112376.

Liu, X., Chen, G., Wu, F., Wen, S., Zuo, W., 2023. Mining top-k high average-utility itemsets based on breadth-first search. Appl. Intell. 53, 29319–29337.

Liu, X., Chen, G., Zuo, W., 2022a. Effective algorithms to mine skyline frequent-utility itemsets. Eng. Appl. Artif. Intell. 116, 105355.

Liu, Y., Liao, W., Choudhary, A., Year. A two-phase algorithm for fast discovery of high utility itemsets. In: Proceedings of the 9th Pacific-Asia Conference on Knowledge Discovery and Data Mining,Hanoi, Vietnam, pp.689-695.

Liu, Z., Ma, Y., Zheng, H., Liu, D., Liu, J., 2022b. Human resource recommendation algorithm based on improved frequent itemset mining. Future Gener. Comput. Syst. 126, 284–288.

Min, F., Zhang, Z.-H., Zhai, W.-J., Shen, R.-P., 2020. Frequent pattern discovery with tri-partition alphabets. Inf. Sci. 507, 715–732.

Nguyen, T.D., Nguyen, L.T., Vu, L., Vo, B., Pedrycz, W., 2021. Efficient algorithms for mining closed high utility itemsets in dynamic profit databases. Expert Syst. Appl. 186, 115741.

Peng, A.Y., Koh, Y.S., Riddle, P., Year mHUIMiner: A fast high utility itemset mining algorithm for sparse datasets. In: Proceedings of the 21st Pacific-Asia Conference on Knowledge Discovery and Data Mining, pp.196-207.

Qu, J.-F., Fournier-Viger, P., Liu, M., Hang, B., Hu, C., 2023. Mining high utility itemsets using prefix trees and utility vectors. IEEE Trans. Knowl. Data Eng. 35, 10224–10236.

Qu, J.-F., Fournier-Viger, P., Liu, M., Hang, B., Wang, F., 2020. Mining high utility itemsets using extended chain structure and utility machine. Knowl.-Based Syst. 208, 106457.

Ryang, H., Yun, U., 2017. Indexed list-based high utility pattern mining with utility upper-bound reduction and pattern combination techniques. Knowl. Inf. Syst. 51, 627–659.

Singh, K., Biswas, B., 2024. Mining top-k high on-shelf utility itemsets using novel threshold raising strategies. ACM Trans. Knowl. Discov. Data 18, 1–23.

Song, W., Sun, Z., Fournier-Viger, P., Wu, Y., 2024. MRI-CE: minimal rare itemset discovery using the cross-entropy method. Inf. Sci. 665, 120392.

Sra, P., Chand, S., 2024a. A reinduction-based approach for efficient high utility itemset mining from incremental datasets. Data Sci. Eng. 9, 73–87.

Sra, P., Chand, S., 2024b. A residual utility-based concept for high-utility itemset mining. Knowl. Inf. Syst. 66, 211–235.

Sun, C., Gong, Y., Guo, Y., Zhao, L., Guan, H., Liu, X., Dong, X., 2024. SN-RNSP: mining self-adaptive nonoverlapping repetitive negative sequential patterns in transaction sequences. Knowl.-Based Syst. 287, 111449.

Truong, T., Duong, H., Le, B., Fournier-Viger, P., Yun, U., 2022. Mining interesting sequences with low average cost and high average utility. Appl. Intell. 52, 7136–7157.

Tseng, V.S., Shie, B.-E., Wu, C.-W., Philip, S.Y., 2012. Efficient algorithms for mining high utility itemsets from transactional databases. IEEE Trans. Knowl. Data Eng. 25, 1772–1786.

Tseng, V.S., Wu, C.W., Shie, B.E., Yu, P.S., Year. UP-Growth: an efficient algorithm for high utility itemset mining. In: Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining,Washington, DC, USA pp.253-262.

Wang, L., Tran, V., Do, T., 2023. A clique-querying mining framework for discovering high utility co-location patterns without generating candidates. ACM Trans. Knowl. Discov. Data 18, 1–42.

Wang, W., Tian, J., Lv, F., Xin, G., Ma, Y., Wang, B., 2021. Mining frequent pyramid patterns from time series transaction data with custom constraints. Comput. Secur. 100, 102088.

Wu, C.-M., Huang, Y.-F., 2011. A cost-efficient and versatile sanitizing algorithm by using a greedy approach. Soft Comput. 15, 939–952.

Wu, J.M.-T., Lin, J.C.-W., Tamrakar, A., 2019. High-utility itemset mining with effective pruning strategies. ACM Trans. Knowl. Discov. Data 13, 1–22.
Wu, P., Niu, X., Fournier-Viger, P., Huang, C., Wang, B., 2022. UBP-Miner: an efficient bit based high utility itemset mining algorithm. Knowl.-Based Syst. 248, 108865.
Yan, Y., Niu, X., Zhang, Z., Fournier-Viger, P., Ye, L., Min, F., 2024. Efficient high utility itemset mining without the join operation. Inf. Sci., 121218
Yang, P., Wang, L., Zhou, L., Chen, H., 2024. A fast spatial high utility co-location pattern mining approach based on branch-and-depth-extension. Inf. Sci. 666, 120407.
Yildirim, I., 2025. An efficient algorithm for fast discovery of high-efficiency patterns. Knowl.-Based Syst. 313, 113157.
Zhang, C., Yang, Y., Du, Z., Gan, W., Yu, P.S., 2023a. HUSP-SP: faster utility mining on sequence data. ACM Trans. Knowl. Discov. Data 18, 1–21.
Zhang, X., Chen, G., Song, L., Gan, W., 2025. Enabling knowledge discovery through low utility itemset mining. Expert Syst. Appl. 265, 125955.
Zhang, X., Chen, G., Song, L., Gan, W., Song, Y., 2023b. HEPM: High-efficiency pattern mining. Knowl.-Based Syst. 281, 111068.
Zhang, Y., Duan, L., Zheng, H., Li-Ling, J., Qin, R., Chen, Z., He, C., Wang, T., 2022. Mining similar aspects for gene similarity explanation based on gene information network. IEEE ACM Trans. Comput. Biol. Bioinf 19, 1734–1746.
Zhu, J., Chen, X., Gan, W., Chen, Z., Yu, P.S., 2025. Targeted mining precise-positioning episode rules. IEEE Transactions on Emerging Topics in Computational Intelligence 9, 904–917.
Zida, S., Fournier-Viger, P., Lin, J.C.-W., Wu, C.-W., Tseng, V.S., 2017. EFIM: a fast and memory efficient algorithm for high-utility itemset mining. Knowl. Inf. Syst. 51, 595–625.